PhAST: Phase-field fracture with differentiable FEM

Release 2026

UKACM Autumn School

Sep 09, 2026

CONTENTS

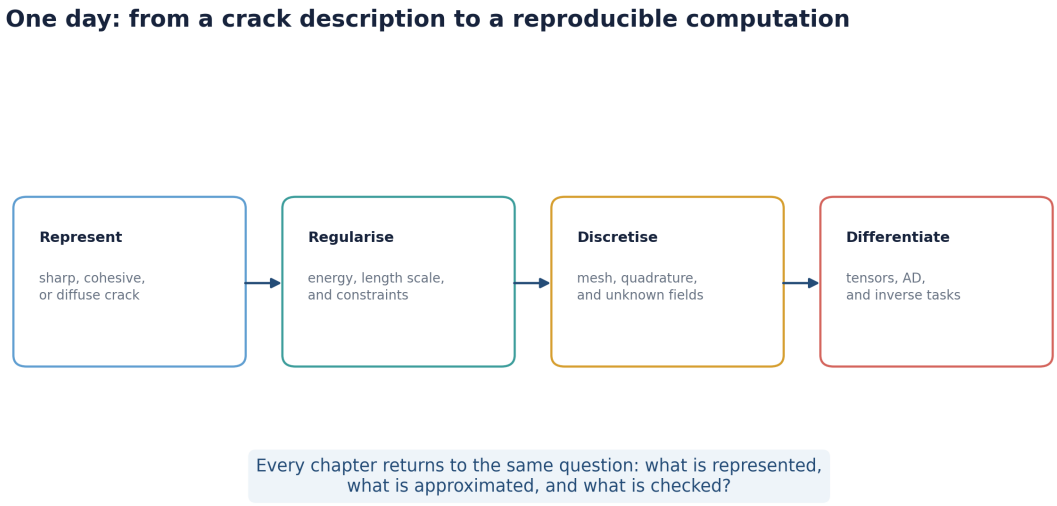


Figure 1: The course route: make each modelling and computational choice visible.

This short textbook develops a reproducible way to reason about phase-field fracture computations. It moves from a stated fracture model to compact computational lessons, retained outputs, exercises, and worked solutions in one reading route. It is not a substitute for a fracture-mechanics text or a production-solver manual. Its goal is narrower and more practical: connect a variational fracture model to a mesh, tensors, differentiation, and honest numerical checks.

The convention throughout is $d = 0$ for intact material and $d = 1$ for fully broken material. A small residual stiffness is retained in practical calculations to avoid a singular elastic operator; it is a numerical device, not a physical claim that a fully cracked material still carries substantial load.

How to use this book

For each topic, make a prediction, read the derivation, inspect the code and its retained output, and answer the exercise before opening the worked solution. The computational lessons are child pages of their theory chapters, not a separate collection of notebook exports. Each may also be downloaded for optional execution in a Jupyter environment.

One lesson contains a small public PhAST quasistatic calculation. The inverse and learning lessons use original teaching models, not fracture-inverse or fracture-acceleration results. When you execute a lesson, record the mesh, parameters, outputs, and observed runtime for that environment. A retained result card never predicts runtime on another machine.

For visual experiments alongside the text, open the interactive explorations. To run the computational lessons, use the environment guide. The printable edition includes the code, recorded figures and complete worked solutions.

CHAPTER

ONE

FROM CRACKS TO COMPUTATION: LEARN BY DOING

1.1 What this course is for

Fracture simulations can look persuasive long before they are understood. A colour map may show a thin damaged band, a reaction curve may soften, and a notebook may differentiate an objective. None of those observations alone identifies the model, the discretisation, or the numerical assumptions that produced them. This course develops a habit of naming each layer.

By the end of the day, you should be able to:

- distinguish a fracture *formulation* from a finite-element *discretisation* and a nonlinear *solution method*;
- write a common phase-field energy and explain the role of its length scale;
- trace a computation from geometry and boundary conditions to arrays, fields, and observable quantities;
- use automatic differentiation for a stated computational graph, and compare one directional derivative with a finite-difference estimate; and
- describe what a learned proposal, adapter, or correction is allowed to claim in a mechanics workflow.

The examples are deliberately small. Their purpose is to make assumptions inspectable, not to establish a material prediction or a benchmark. Each computational lesson follows the same rhythm: predict, derive or identify the relevant quantity, inspect the code and retained output, then test your interpretation against a worked solution.

1.2 Opening experiment: can a good average be a bad prediction?

Suppose an input admits two valid states. Will a single learned average be valid too? The short opening experiment compares balanced supervised fitting with an explicitly selected energy-minimising branch. Predict the answer before opening the plots; return to its gradient calculation after the differentiation chapter.

1.2.1 Why average predictions can fail

Learning objective. Use an original reduced quartic-energy toy to distinguish a balanced supervised conditional mean from an explicitly selected positive energy branch, and identify a stationary-gradient failure mode.

Predict first. For equally likely targets $+\sqrt{s}$ and $-\sqrt{s}$ at the same control s, predict the best single squared-error prediction. Then predict whether $q = 0$ must have a useful energy gradient whenever its residual is nonzero.

Recorded computation: 6.381 seconds on the reference local CPU after setup. This is not a fresh Colab timing.

Read the code and its recorded outputs in order. To change inputs and run Python, download a notebook and use the course environment. The static book does not execute these cells in the browser.

Download practice notebook · Download with worked solutions · Environment setup

This is an original, reduced **PyTorch algebraic toy** for the course opening. For a control value $s \in [0.25, 1]$, define

$$W(q; s) = \frac{(q^2 - s)^2}{4}.$$

Each $s > 0$ has two zero-energy reference branches, $q_+(s) = +\sqrt{s}$ and $q_-(s) = -\sqrt{s}$. We compare two objectives:

- a supervised squared-error model trained on *balanced* samples from both branches, whose conditional target mean is near $q = 0$;

- a residual/energy-trained model with an **explicit positive branch selection**, $q = \mathrm{softplus}(a) > 0$.

This is not PhAST fracture, a solver-in-the-loop experiment, or a generative model that covers both modes. It only makes a compact point: multimodal targets, objective choice, and branch selection matter.

```
from pathlib import Path
import json
import platform
import time

import matplotlib.pyplot as plt
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(20260909)
torch.set_num_threads(1)
dtype = torch.float64

def locate_course_root():
    for base in (Path.cwd(), *Path.cwd().parents):
        direct = base / "teaching" / "ukacm_autumn_school_2026"
        if (direct / "assets").is_dir():
            return direct
        if (base / "notebooks").is_dir() and (base / "assets").is_dir():
            return base
    raise FileNotFoundError("Could not find the UKACM course root.")

COURSE_ROOT = locate_course_root()
ASSETS = COURSE_ROOT / "assets" / "teaser"
ASSETS.mkdir(parents=True, exist_ok=True)
print({
    "scope": "original reduced algebraic energy toy; not PhAST fracture or a␣
↪solver-in-loop result",
    "torch": torch.__version__,
    "device": "cpu",
    "dtype": str(dtype),
    "threads": torch.get_num_threads(),
})
```

```
{'scope': 'original reduced algebraic energy toy; not PhAST fracture or a solver-
↪in-loop result', 'torch': '2.8.0', 'device': 'cpu', 'dtype': 'torch.float64',
↪'threads': 1}
```

Balanced supervision versus an energy objective

The training inputs are repeated so that every s has one positive and one negative target. Under squared error, the best single deterministic prediction is their conditional mean, 0. That point is between the two reference branches and has nonzero residual $q^2 - s = -s$.

```
s_unique = torch.linspace(0.25, 1.00, 96, dtype=dtype).unsqueeze(1)
branch_positive = torch.sqrt(s_unique)
branch_negative = -branch_positive
s_supervised = s_unique.repeat_interleave(2, dim=0)
q_supervised_target = torch.stack(
    (branch_positive.squeeze(1), branch_negative.squeeze(1)), dim=1
).reshape(-1, 1)
s_heldout = torch.linspace(0.255, 0.995, 73, dtype=dtype).unsqueeze(1)

def residual(q, s):
    return q.square() - s
```

(continues on next page)

(continued from previous page)

```
def energy(q, s):
    return residual(q, s).square() / 4.0

class TinyScalarMLP(nn.Module):
    def __init__(self, positive_branch=False):
        super().__init__()
        self.hidden = nn.Sequential(
            nn.Linear(1, 16),
            nn.Tanh(),
            nn.Linear(16, 16),
            nn.Tanh(),
            nn.Linear(16, 1),
        )
        self.positive_branch = positive_branch

    def forward(self, s):
        raw = self.hidden(s)
        return F.softplus(raw) + 1.0e-8 if self.positive_branch else raw

print({
    "unique_s_train": int(s_unique.shape[0]),
    "balanced_supervised_pairs": int(s_supervised.shape[0]),
    "heldout_s": int(s_heldout.shape[0]),
    "reference_branches_per_s": 2,
})
```

```
{'unique_s_train': 96, 'balanced_supervised_pairs': 192, 'heldout_s': 73,
↪'reference_branches_per_s': 2}
```

```
torch.manual_seed(20260909)
supervised_model = TinyScalarMLP().to(dtype=dtype)
supervised_optimiser = torch.optim.Adam(supervised_model.parameters(), lr=2.0e-2)
supervised_start = time.perf_counter()
for _ in range(800):
    supervised_optimiser.zero_grad(set_to_none=True)
    supervised_loss = torch.mean(
        (supervised_model(s_supervised) - q_supervised_target).square()
    )
    supervised_loss.backward()
    supervised_optimiser.step()
supervised_seconds = time.perf_counter() - supervised_start

torch.manual_seed(20260910)
positive_model = TinyScalarMLP(positive_branch=True).to(dtype=dtype)
positive_optimiser = torch.optim.Adam(positive_model.parameters(), lr=2.0e-2)
positive_start = time.perf_counter()
for _ in range(1000):
    positive_optimiser.zero_grad(set_to_none=True)
    positive_loss = energy(positive_model(s_unique), s_unique).mean()
    positive_loss.backward()
    positive_optimiser.step()
positive_seconds = time.perf_counter() - positive_start

print({
    "supervised_mse_on_balanced_targets": float(supervised_loss.detach()),
    "energy_loss_positive_branch": float(positive_loss.detach()),
    "supervised_training_seconds": round(supervised_seconds, 4),
    "positive_branch_training_seconds": round(positive_seconds, 4),
})
```

```
{'supervised_mse_on_balanced_targets': 0.6250000049495941, 'energy_loss_positive_
↪branch': 1.6824936103105914e-06, 'supervised_training_seconds': 0.2126,
↪'positive_branch_training_seconds': 0.2239}
```

A stationary point can still be wrong

The energy derivative is

$$\frac{\partial W}{\partial q} = q(q^2 - s).$$

Thus $q = 0$ has zero energy gradient even though its residual is $-s$ and its energy is positive when $s > 0$. The next cell evaluates that fact directly. The successful energy model above avoids a zero-output start by selecting the positive branch with a softplus output; it does not discover both branches.

```
s_stationary = torch.tensor(0.64, dtype=dtype)
q_stationary = torch.tensor(0.0, dtype=dtype, requires_grad=True)
W_stationary = energy(q_stationary, s_stationary)
(grad_stationary,) = torch.autograd.grad(W_stationary, q_stationary)
r_stationary = residual(q_stationary.detach(), s_stationary)
print({
    "s": float(s_stationary),
    "W_at_q0": float(W_stationary.detach()),
    "residual_at_q0": float(r_stationary),
    "dW_dq_at_q0": float(grad_stationary),
})
assert abs(float(grad_stationary)) < 1.0e-15
assert abs(float(r_stationary)) > 0.1
```

```
{'s': 0.64, 'W_at_q0': 0.1024, 'residual_at_q0': -0.64, 'dW_dq_at_q0': -0.0}
```

```
with torch.no_grad():
    q_supervised = supervised_model(s_heldout)
    q_positive = positive_model(s_heldout)
    r_supervised = residual(q_supervised, s_heldout)
    r_positive = residual(q_positive, s_heldout)
    W_supervised = energy(q_supervised, s_heldout)
    W_positive = energy(q_positive, s_heldout)
    metrics = {
        "heldout_count": int(s_heldout.shape[0]),
        "supervised_max_abs_prediction": float(q_supervised.abs().max()),
        "supervised_mean_abs_residual": float(r_supervised.abs().mean()),
        "supervised_mean_energy": float(W_supervised.mean()),
        "positive_min_prediction": float(q_positive.min()),
        "positive_mean_abs_residual": float(r_positive.abs().mean()),
        "positive_max_abs_residual": float(r_positive.abs().max()),
        "positive_mean_energy": float(W_positive.mean()),
        "positive_branch_rmse": float(
            torch.sqrt(torch.mean((q_positive - torch.sqrt(s_heldout)).square()))
        ),
    }

assert metrics["supervised_max_abs_prediction"] < 0.03
assert metrics["supervised_mean_abs_residual"] > 0.50
assert metrics["positive_min_prediction"] > 0.0
assert metrics["positive_mean_abs_residual"] < 0.02
assert metrics["positive_mean_energy"] < metrics["supervised_mean_energy"]
print(metrics)
(ASSETS / "teaser_metrics.json").write_text(
    json.dumps(metrics, indent=2), encoding="utf-8"
)
```

```
{'heldout_count': 73, 'supervised_max_abs_prediction': 0.0001623609215912064,
↪'supervised_mean_abs_residual': 0.6249999952779399, 'supervised_mean_energy': 0.
↪10938147994308955, 'positive_min_prediction': 0.5136183526070731, 'positive_mean_
↪abs_residual': 0.002002033038503439, 'positive_max_abs_residual': 0.
↪00880381213480369, 'positive_mean_energy': 1.5331257738901223e-06, 'positive_
↪branch_rmse': 0.001987931297689105}
```

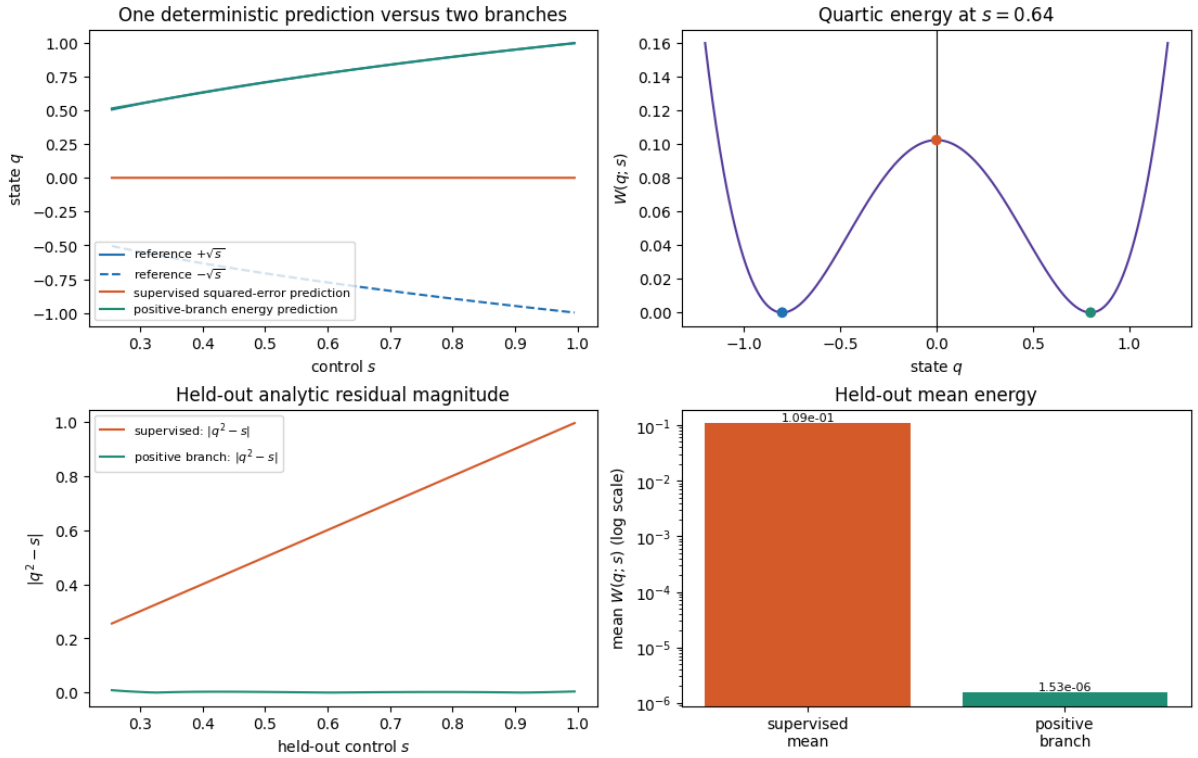
```
439
```

```
s_plot = s_heldout.squeeze(1).cpu().numpy()
q_sup_plot = q_supervised.squeeze(1).cpu().numpy()
q_pos_plot = q_positive.squeeze(1).cpu().numpy()
r_sup_plot = r_supervised.abs().squeeze(1).cpu().numpy()
r_pos_plot = r_positive.abs().squeeze(1).cpu().numpy()
s_energy = torch.tensor(0.64, dtype=dtype)
q_axis = torch.linspace(-1.2, 1.2, 500, dtype=dtype)

fig, axes = plt.subplots(2, 2, figsize=(11, 7), constrained_layout=True)
axes[0, 0].plot(s_plot, torch.sqrt(s_heldout).squeeze(1).cpu().numpy(),
                color="#2070b4", label=r"reference $+\sqrt{s}$")
axes[0, 0].plot(s_plot, -torch.sqrt(s_heldout).squeeze(1).cpu().numpy(),
                color="#2070b4", linestyle="--", label=r"reference $-\sqrt{s}$")
axes[0, 0].plot(s_plot, q_sup_plot, color="#d45a2a",
                label="supervised squared-error prediction")
axes[0, 0].plot(s_plot, q_pos_plot, color="#218c74",
                label="positive-branch energy prediction")
axes[0, 0].set(title="One deterministic prediction versus two branches",
               xlabel=r"control $s$", ylabel=r"state $q$")
axes[0, 0].legend(fontsize=8, loc="best")

axes[0, 1].plot(q_axis.cpu().numpy(), energy(q_axis, s_energy).cpu().numpy(),
                color="#553c9a")
axes[0, 1].scatter([0.0, float(torch.sqrt(s_energy)), -float(torch.sqrt(s_
↪energy))],
                   [float(energy(torch.tensor(0.0, dtype=dtype), s_energy)),
                    0.0, 0.0],
                   color=["#d45a2a", "#218c74", "#2070b4"], zorder=3)
axes[0, 1].axvline(0.0, color="black", linewidth=0.8)
axes[0, 1].set(title=r"Quartic energy at $s=0.64$",
               xlabel=r"state $q$", ylabel=r"$W(q;s)$")

axes[1, 0].plot(s_plot, r_sup_plot, color="#d45a2a",
                label="supervised: $|q^2-s|$")
axes[1, 0].plot(s_plot, r_pos_plot, color="#218c74",
                label="positive branch: $|q^2-s|$")
axes[1, 0].set(title="Held-out analytic residual magnitude",
               xlabel=r"held-out control $s$", ylabel=r"$|q^2-s|$")
axes[1, 0].legend(fontsize=8)

names = ["supervised\nmean", "positive\nbranch"]
axes[1, 1].bar(names, [metrics["supervised_mean_energy"], metrics["positive_mean_
↪energy"]],
               color=["#d45a2a", "#218c74"])
axes[1, 1].set_yscale("log")
axes[1, 1].set(title="Held-out mean energy",
               ylabel=r"mean $W(q;s)$ (log scale)")
for index, value in enumerate(
    [metrics["supervised_mean_energy"], metrics["positive_mean_energy"]]
):
    axes[1, 1].text(index, value, f"{value:.2e}", ha="center", va="bottom",␣
↪fontsize=8)
```

(continues on next page)

(continued from previous page)

```
figure_path = ASSETS / "why_average_predictions_can_fail.png"
fig.savefig(figure_path, dpi=180, bbox_inches="tight")
plt.show()
plt.close(fig)
print({"figure": str(figure_path.relative_to(COURSE_ROOT)), "metrics": metrics})
```




```
{'figure': 'assets/teaser/why_average_predictions_can_fail.png', 'metrics': {
↪'heldout_count': 73, 'supervised_max_abs_prediction': 0.0001623609215912064,
↪'supervised_mean_abs_residual': 0.6249999952779399, 'supervised_mean_energy': 0.
↪10938147994308955, 'positive_min_prediction': 0.5136183526070731, 'positive_mean_
↪abs_residual': 0.002002033038503439, 'positive_max_abs_residual': 0.
↪00880381213480369, 'positive_mean_energy': 1.5331257738901223e-06, 'positive_
↪branch_rmse': 0.001987931297689105}}
```


Interpret the teaser

The supervised model is close to the balanced conditional mean, which is near $q = 0$ and is not one of the two zero-energy branches for $s > 0$. The energy-trained model is useful here because the positive softplus output explicitly commits to one branch and starts away from the stationary $q = 0$ point. It represents **one** positive branch only.

This example does not show generative multimodal coverage, PhAST phase-field fracture, a residual from an FE calculation, solver-in-loop differentiation, or a performance gain. It also shows a limitation: a nonzero residual can coexist with a zero quartic-energy gradient at $q = 0$, so a physics loss is not automatically an optimisation success.

Exercises

Try each question before opening the hint or worked solution. Keep the original calculation as your reference.

Exercise 1: Derive the mean prediction and the stationary trap

At a fixed s, let $a = \sqrt{s}$ and suppose the two supervised targets $+a$ and $-a$ are equally represented. The reduced energy is

$$W(q; s) = \frac{(q^2 - s)^2}{4}.$$

1. Show that the balanced squared-error objective has its single deterministic minimiser at $q = 0$.
2. At $s = 0.64$, compute the residual $r = q^2 - s$, the energy W, and $\partial W/\partial q$ at $q = 0$.
3. Explain why the last result is a warning about optimisation rather than evidence that $q = 0$ is a valid reference branch.

Hint 1

Average the two squared errors before differentiating. For the energy, use $\partial W/\partial q = q(q^2 - s)$.

Worked solution 1

1. The balanced squared-error objective is

$$L(q) = \frac{1}{2}\left[(q - a)^2 + (q + a)^2\right] = q^2 + a^2.$$

Thus $\partial L/\partial q = 2q$, and its unique minimiser is $q = 0$. For $s > 0$, this lies between the two reference branches $\pm\sqrt{s}$ rather than on either one.

2. With $s = 0.64$ and $q = 0$,

$$r = -0.64, \qquad W = \frac{(-0.64)^2}{4} = 0.1024, \qquad \frac{\partial W}{\partial q} = 0(-0.64) = 0.$$

These are exactly the retained notebook values.

3. The residual and energy are nonzero, but the quartic energy has a stationary point at $q = 0$. A gradient-based energy objective can therefore fail to move from that exact point. The positive-branch model avoids this particular start by imposing $q = \mathrm{softplus}(a) > 0$; it does not make the middle point an admissible branch.

Exercise 2: Interpret the held-out branch metrics

The deterministic supervised model has held-out maximum absolute prediction 1.6236×10^{-4}, mean absolute residual 0.6249999953, and mean energy 0.1093814799. The explicitly positive energy model has minimum prediction 0.5136183526, mean absolute residual 0.0020020330, maximum residual 0.0088038121, mean energy 1.5331×10^{-6}, and positive-branch RMSE 0.0019879313.

1. Which model is closer to the positive zero-energy branch on held-out s, and what metric supports that conclusion?
2. Why is the supervised prediction near zero despite its poor residual?
3. State two things this result does **not** demonstrate.

Hint 2

A zero-energy branch satisfies $q^2 - s = 0$. Compare the residual and energy before interpreting the sign or model class.

Worked solution 2

1. The explicitly positive energy model is closer to the positive branch. Its mean absolute residual is about 0.0020, compared with about 0.6250 for the supervised mean, and its held-out mean energy is 1.5331×10^{-6} rather than 1.0938×10^{-1}. Its positive-branch RMSE is also only about 0.00199.
2. The targets at each s are exactly balanced between positive and negative branches. Squared-error regression therefore estimates their conditional mean, which is 0. That value can minimise the supervised loss while failing the algebraic branch relation $q^2 = s$.
3. This is not generative multimodal coverage: the positive model intentionally represents one branch only. It is also not PhAST phase-field fracture, an FE residual, a solver-in-loop differentiability result, or a speed comparison. The lesson isolates how target distribution, objective, branch selection, and local gradient geometry change the outcome.

This original algebraic activity is inspired by the Physics-based Deep Learning teaser. It is not a phase-field fracture model. It introduces a recurring question: does the objective measure the property that we actually want?

1.3 Reading and running the book

The primary route is contained in this book. Read a theory section, continue to its computational lesson in the local table of contents, and return to the next theory section with one observation in hand.

Read and predict. Read the figures and worked equations first. Before a code cell, state what sign, field pattern, or check you expect to see.

Inspect and explain. The five integrated lessons pair the reviewed code with its retained figures and result cards: a degradation derivative, a public PhAST notched-tension calculation, a differentiable elastic-bar toy, a saved toy field model, and a checked toy proposal with fallback. Use the exercise and solution in each lesson to distinguish what the output shows from what it does not establish.

Execute optionally. Download a lesson notebook only when you want to alter an input or repeat the calculation in a Jupyter environment. Read its recorded result card before running it. The record belongs to the environment in which it was measured; it is not a portable runtime prediction.

Extend a local solver deliberately. Install and document a solver environment only when you need to alter the physics, mesh, or algorithm. The public PhAST source repository is a useful starting point for source and release information. A local run should record the software revision, numerical settings, and output location alongside the physical parameters.

1.4 The result card

For every computational activity, write a compact result card. It separates what was specified from what was observed.

1.4.1 Minimum result card

1. **Case:** geometry, material regions, and boundary/loading conditions.
2. **Numerics:** mesh or nodal layout, quadrature convention, load steps, tolerances, and the phase-field length ℓ.
3. **Environment:** software version, device, precision, and seed where a random process is used.
4. **Observation:** the selected field, scalar quantity, and a labelled plot.
5. **Check:** a named equilibrium, residual, constraint, or derivative check.
6. **Limit:** one sentence describing what the check does *not* establish.

For example, a monotone increase in d at one node is consistent with a damage-irreversibility rule, but it does not by itself demonstrate mesh objectivity. A close directional derivative match at one parameter value checks a local calculation, not the uniqueness of an inverse recovery.

1.5 A vocabulary for careful claims

Use the following three sentence patterns when describing a numerical result.

- **Specified:** "The calculation used $d = 0$ as intact, $d = 1$ as broken, a stated regularisation length ℓ, and the listed boundary conditions."
- **Observed:** "For this discretisation and load increment, the plotted damage field localised near the prescribed notch."
- **Not inferred:** "This observation alone does not identify a physical crack path at all mesh resolutions or loading rates."

This is not cautious wording for its own sake. It tells another reader which part of a conclusion they can reproduce, challenge, or extend.

1.6 A first prediction before a first computational lesson

Before reading the first computational lesson, make a prediction in words. If the tensile energy driving damage is high near a notch and the loading increases, where should a phase-field model first permit the damage variable to grow? What quantity would you plot to distinguish a narrow diffuse band from a broadly distributed reduction in stiffness?

After the run, return to the prediction. A useful explanation has four parts:

1. the location and shape actually observed;
2. the terms in the energy that encourage or resist that shape;
3. the numerical choice that may influence it; and
4. one test that would make the explanation stronger.

This pattern will be used throughout the book.

1.7 Exercise: label the layers

Classify each statement as a fracture formulation, spatial discretisation, solution/sensitivity method, or observable.

1. "Use a scalar damage field and a gradient penalty."
2. "Add a discontinuous enrichment near a crack."
3. "Alternate displacement and damage subproblems."
4. "Compare reaction force against imposed displacement."
5. "Differentiate a scalar loss with respect to a material multiplier."

Solution

1. A phase-field **fracture formulation**. The scalar field and gradient penalty define a regularised crack representation.
2. **Spatial discretisation**: this describes the central idea of XFEM.
3. A **solution method**: staggered solution can be applied to a chosen formulation and discretisation.
4. An **observable** derived from the computed state and boundary data.
5. A **sensitivity method/task**. Automatic differentiation or an adjoint may compute the derivative; neither is itself a fracture formulation.

1.8 What to carry into Chapter 1

Keep two questions visible: *what object represents the crack?* and *what exactly is solved?* The next chapter places phase field, XFEM, and cohesive models next to one another without treating them as interchangeable labels.

CHAPTER

TWO

CRACK REPRESENTATIONS: SHARP, COHESIVE, AND DIFFUSE

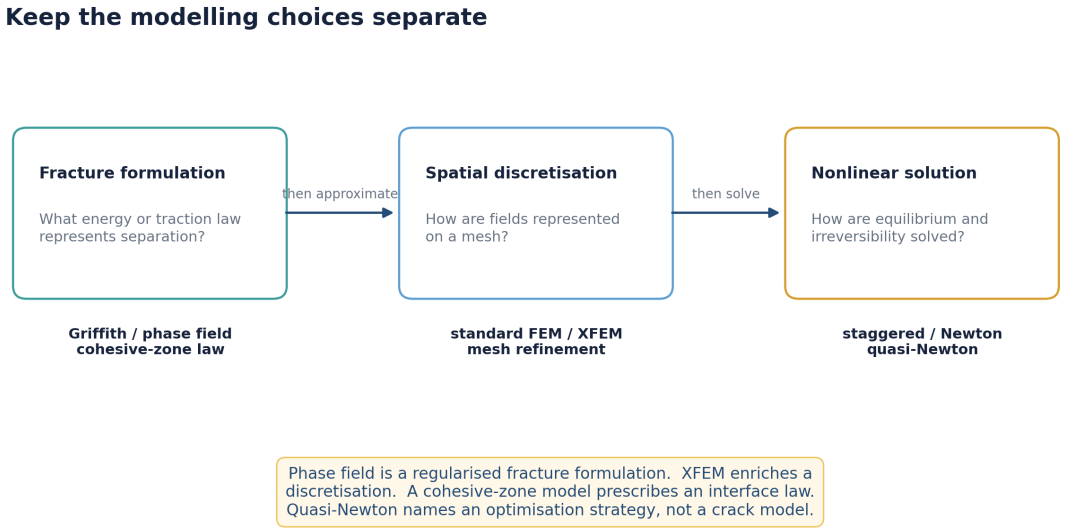


Figure 2.1: Three different questions are often compressed into one phrase such as “the fracture method.” Keeping them separate prevents misleading comparisons.

2.1 The starting point: a sharp crack

In classical brittle fracture, a crack is a lower-dimensional set Γ inside a body Ω. A Griffith-type idealisation balances bulk elastic energy, external work, and an energy proportional to newly created crack surface:

$$\mathcal{E}(u,\Gamma) = \int_{\Omega\setminus\Gamma} \psi(\varepsilon(u))\,\mathrm{d}x + G_c\,\mathcal{H}^{d-1}(\Gamma) - \mathcal{W}_{\mathrm{ext}}(u).$$

Here u is displacement, ψ is elastic energy density, G_c is fracture toughness, and $\mathcal{H}^{d-1}(\Gamma)$ measures crack length in two dimensions or area in three. The attractive feature is conceptual clarity: the model makes a literal discontinuity. The difficult feature is that the unknown crack geometry may advance, branch, or meet another crack during the calculation.

The energy criterion is associated with Griffith (1921) and the variational treatment of evolving cracks by Francfort and Marigo (1998).

2.2 Cohesive-zone models: separation on an interface

A cohesive-zone model (CZM) places a constitutive traction–separation law on an interface. If δ denotes displacement jump, a simple scalar picture is

$$T = T(\delta), \qquad \Phi(\delta) = \int_0^\delta T(s)\,\mathrm{d}s.$$

The area below $T(\delta)$ supplies an interface fracture energy. A CZM is especially natural when an interface is known in advance: an adhesive layer, a ply interface, or a prescribed material boundary. Its parameters can express a peak traction and a separation scale in addition to an energy.

The interface still needs to be represented numerically. It may coincide with element faces, embedded interface elements, or a tracked surface. A cohesive law is therefore a **fracture formulation or constitutive law**, not a particular mesh technology. The classical computational formulation of Xu and Needleman (1994) is a standard reference.

Useful boundary

It is misleading to say that a CZM "is XFEM" or that it "is phase field." A cohesive law can be coupled to several discretisations; it has a different physical idealisation from a diffuse crack-density regularisation.

2.3 XFEM: enrich an approximation space

The extended finite-element method (XFEM) represents a discontinuity or a near-tip feature by enriching a conventional finite-element approximation. A schematic displacement approximation is

$$u_h(x) = \sum_{i\in I} N_i(x)u_i + \sum_{j\in J} N_j(x)H(x)a_j + \sum_{k\in K} N_k(x)\sum_{\alpha} F_\alpha(x)b_{k\alpha}.$$

N_i are ordinary shape functions; H may encode a jump across a crack; and F_α can encode near-tip asymptotic behaviour. The index sets J and K select enriched nodes. The important point is not this exact notation but its role: XFEM changes the **spatial approximation**, allowing a crack to cut through elements without forcing every element edge to align with the crack.

XFEM can reduce repeated remeshing for propagating cracks, but brings its own implementation questions: enrichment support, integration of cut cells, conditioning, branch handling, and the geometry used to describe the crack. The foundational paper is Moes, Dolbow, and Belytschko (1999).

2.4 Phase field: replace a surface by a narrow band

In a phase-field formulation, a scalar field $d(x)$ spreads the crack over a band of width governed by a length ℓ. With the convention used here, $d = 0$ is intact and $d = 1$ is broken. A typical crack-density functional is

$$\Gamma_\ell(d) = \frac{1}{c_0}\int_\Omega \left(\frac{w(d)}{\ell} + \ell|\nabla d|^2\right)\mathrm{d}x, \qquad c_0 = 4\int_0^1 \sqrt{w(s)}\,\mathrm{d}s.$$

As ℓ becomes small under appropriate refinement and modelling assumptions, this regularised functional approximates a sharp fracture surface. The method avoids explicitly tracking a discontinuity during propagation. It instead solves for a continuous field d and must resolve its diffuse band on the mesh.

This is a **regularised fracture formulation**. It is commonly discretised with ordinary continuous finite elements, but the formulation and the discretisation are conceptually separate. The diffuse approach was developed for brittle fracture by Bourdin, Francfort, and Marigo (2000).

2.5 A fair comparison

Choose a method by first stating the question.

If the crack path and interface are known. An interface model with a cohesive law may make the physical parameters particularly direct. The price is that a new path outside that interface is not automatically represented.

If a sharp discontinuity is needed on a fixed background mesh. XFEM gives a route to enrich the approximation. It does not by itself choose a fracture criterion or resolve a crack evolution law.

If evolving, branching, or merging cracks make explicit tracking awkward. A phase field can be convenient because the crack is an evolving field. Resolution and regularisation sensitivity then become central scientific questions rather than implementation details.

These statements compare *useful roles*, not universal winners. Cost depends on mesh, dimension, nonlinear solver, loading path, conditioning, and the question being asked.

2.6 Method and algorithm are not synonyms

The following labels belong at different levels:

- **phase field, Griffith, cohesive zone:** fracture energy or interface-law choices;
- **standard FEM, XFEM, mesh refinement:** spatial representations;
- **staggered iteration, Newton, quasi-Newton:** nonlinear solution methods;
- **finite difference, unrolled automatic differentiation, implicit adjoint:** sensitivity strategies.

For example, a phase-field calculation can use standard finite elements and a staggered solver; another may use Newton-type iterations. A quasi-Newton update is not a third crack representation.

2.7 Exercise: separate the layers before reading code

Rewrite the sentence below as two technically accurate sentences.

> "We use XFEM phase field with a quasi-Newton fracture method."

Solution

One possible rewrite is: "We represent brittle fracture with a phase-field regularisation and approximate its displacement and damage fields with a stated finite-element space. We solve the resulting nonlinear problem with a quasi-Newton method; XFEM would be relevant only if the approximation space is explicitly enriched for a discontinuity or crack-tip feature."

The original sentence may describe a legitimate hybrid method, but it does not say which component provides the fracture energy, which provides enrichment, or which equation the optimisation method solves.

2.8 Take-away

Before comparing curves, mesh sizes, or runtimes, identify the crack representation, the finite-element approximation, and the nonlinear solver. That separation makes the phase-field energy in the next chapter much easier to read. It also gives a useful template for the later computational lessons: the case card states the formulation, mesh and solver route before any output is interpreted.

CHAPTER

THREE

FRACTURE ENERGY: AT1, AT2, DEGRADATION, AND WIDTH

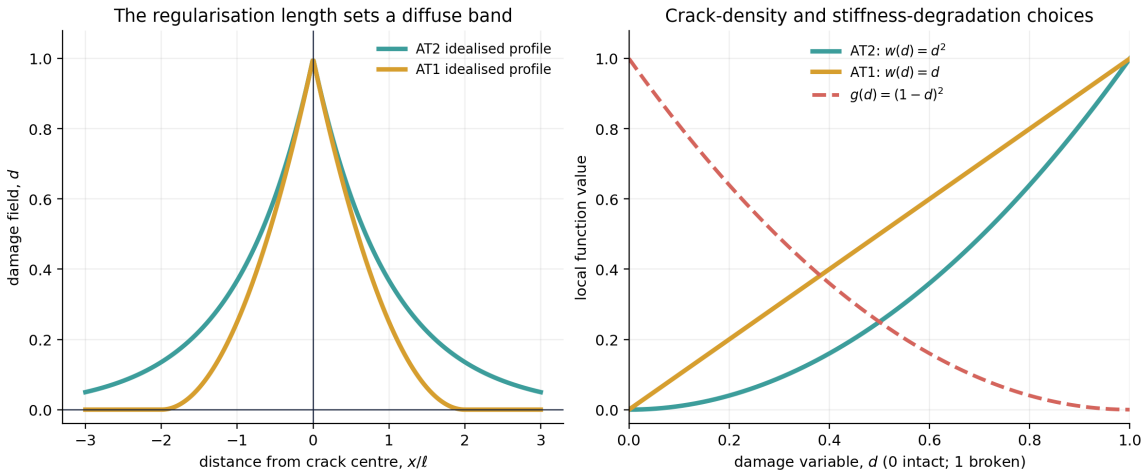


Figure 3.1: The crack-density choice and the degradation law are separate functions. The curves shown are mathematical profiles, not measured material data.

3.1 A common energy template

For small-strain brittle fracture, a frequently used phase-field energy is

$$\mathcal{E}_\ell(u,d) = \int_\Omega \left[g_\eta(d)\,\psi^+(\varepsilon(u)) + \psi^-(\varepsilon(u)) + \frac{G_c}{c_0}\left(\frac{w(d)}{\ell} + \ell|\nabla d|^2\right)\right] \mathrm{d}x - \mathcal{W}_{\text{ext}}(u).$$

The ingredients have different jobs:

- ψ^+ is the part of elastic energy selected to drive fracture, often a tensile contribution;
- ψ^- is retained rather than degraded in a tension–compression split;
- $g_\eta(d)$ reduces tensile stiffness as damage grows;
- G_c sets the energy scale of fracture;
- $w(d)$ selects a crack-density family; and
- ℓ controls the regularised width of the damage band.

This notation does not identify a unique model. Different spectral, volumetric–deviatoric, or other decompositions define ψ^+ and ψ^- differently. State the split before interpreting compression, crack closure, or mixed-mode behaviour.

3.2 The damage convention and degradation derivative

This book uses $d = 0$ for intact material and $d = 1$ for fully broken material. A standard residual-stiffness degradation law is

$$g_\eta(d) = (1 - \eta_{\text{res}})(1 - d)^2 + \eta_{\text{res}}, \qquad 0 < \eta_{\text{res}} \ll 1.$$

It satisfies $g_\eta(0) = 1$ and $g_\eta(1) = \eta_{\text{res}}$. The residual term helps prevent the elastic operator from becoming singular in a damaged zone. It should be reported because it affects the numerical problem and the post-peak response.

For a hand calculation or a tensor implementation, differentiate explicitly:

$$g'_\eta(d) = -2(1 - \eta_{\text{res}})(1 - d), \qquad g''_\eta(d) = 2(1 - \eta_{\text{res}}).$$

The negative first derivative is meaningful: increasing d lowers the degraded tensile energy. It does *not* mean that damage is free to grow everywhere, because fracture density, gradients, boundary data, and irreversibility all enter the total variation.

3.3 Computational lesson: inspect a degradation derivative

Continue directly to the lesson below. Predict the sign of $g'_\eta(d)$ at an interior damage value, then compare the analytic derivative with automatic differentiation and a central finite difference. The code calls the pinned public PhAST material law for this *local, smooth* check. It does not differentiate a full coupled fracture history, active constraint, or nonlinear solve.

3.3.1 Differentiate a degradation law

Learning objective. Work with the exact normalised quadratic degradation law in the pinned public PhAST material object, and interpret an analytic/autograd/finite-difference agreement at the right scope.

Predict first. Predict whether the residual parameter $\eta = 10^{-7}$ makes $g(1)$ exactly zero, and whether a very small local finite-difference error alone proves a full coupled fracture calculation is differentiable.

Recorded computation: 4.721 seconds on the reference local CPU after setup. This is not a fresh Colab timing.

Read the code and its recorded outputs in order. To change inputs and run Python, download a notebook and use the course environment. The static book does not execute these cells in the browser.

Download practice notebook · Download with worked solutions · Environment setup

This short exercise checks the exact standard normalised degradation law in the pinned public PhAST source:

$$g(d) = (1 - \eta)(1 - d)^2 + \eta, \qquad g'(d) = -2(1 - \eta)(1 - d).$$

We use `d=0` for intact and `d=1` for broken, `eta=10^{-7}`, and `float64`. The comparison is deliberately local and smooth: it is not a claim that every coupled fracture route is automatically differentiable through history updates, convergence logic, clamps, or active sets.

```
from pathlib import Path
import sys

def locate_course_root():
    for base in (Path.cwd(), *Path.cwd().parents, Path(__file__).resolve().parent␣
↪if "__file__" in globals() else Path.cwd()):
        direct = base / "teaching" / "ukacm_autumn_school_2026"
        if (direct / "notebooks" / "day2_helpers").is_dir():
            return direct
        if (base / "notebooks" / "day2_helpers").is_dir():
            return base
    raise FileNotFoundError("Could not find teaching/ukacm_autumn_school_2026.")
```

(continues on next page)

(continued from previous page)

```
COURSE_ROOT = locate_course_root()
if str(COURSE_ROOT / "notebooks") not in sys.path:
    sys.path.insert(0, str(COURSE_ROOT / "notebooks"))

import matplotlib.pyplot as plt
import numpy as np
import torch
from io import BytesIO
from IPython.display import Image, display

from day2_helpers import assets_dir

torch.set_default_dtype(torch.float64)

def display_figure(figure, dpi=150, alt="Rendered teaching figure"):
    """Embed a PNG even when nbclient uses a non-interactive Agg backend."""
    buffer = BytesIO()
    figure.savefig(buffer, format="png", dpi=dpi, bbox_inches="tight")
    display(Image(data=buffer.getvalue(), alt=alt))

print({"course_layout": "teaching/ukacm_autumn_school_2026 located", "torch":␣
↪torch.__version__, "device": "cpu", "dtype": str(torch.get_default_dtype())})
```

```
{'course_layout': 'teaching/ukacm_autumn_school_2026 located', 'torch': '2.8.0',
↪'device': 'cpu', 'dtype': 'torch.float64'}
```

```
from day2_helpers import import_public_phast
phast_info = import_public_phast()
from phast.physics.material import Material

eta = 1.0e-7
material = Material(E=1.0, nu=0.3, Gc=1.0, l0=0.1, rho=1.0, eta_residual=eta)

def g_analytic(d):
    return (1.0 - eta) * (1.0 - d) ** 2 + eta

def gp_analytic(d):
    return -2.0 * (1.0 - eta) * (1.0 - d)

d = torch.tensor(0.25, dtype=torch.float64, requires_grad=True)
g_phast = material.degradation(d)
(g_ad,) = torch.autograd.grad(g_phast, d)
h = 1.0e-6
g_fd = (g_analytic(d.detach() + h) - g_analytic(d.detach() - h)) / (2.0 * h)
expected = gp_analytic(d.detach())
print({
    "public_source": phast_info,
    "d": float(d.detach()),
    "g_from_public_material": float(g_phast.detach()),
    "analytic_g_prime": float(expected),
    "autograd_g_prime": float(g_ad),
    "central_fd_g_prime": float(g_fd),
})
```

```
{'public_source': {'source': 'vendor/PhAST/src (manifest-verified when vendored)',
↪'module_file': 'phast/__init__.py', 'revision':
↪'f6324f899f0701769810be117f27f1208f7a582e', 'version': '0.16.2', 'import_route':
↪'PYTHONPATH=<public-PhAST>/src', 'verification': 'COURSE_SOURCE_MANIFEST.json␣
↪hashes verified'}, 'd': 0.25, 'g_from_public_material': 0.56250004375, 'analytic_
↪g_prime': -1.49999985, 'autograd_g_prime': -1.49999985, 'central_fd_g_prime': -1.
```

(continues on next page)

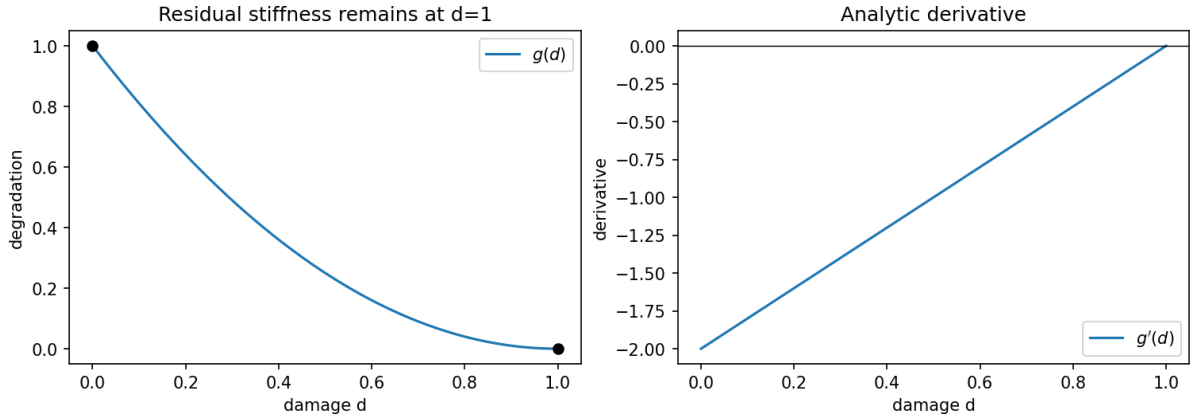
(continued from previous page)

```
↪4999998499964917}
```

```
endpoints = material.degradation(torch.tensor([0.0, 1.0], dtype=torch.float64))
ad_error = abs(float(g_ad - expected))
fd_error = abs(float(g_fd - expected))
assert abs(float(endpoints[0]) - 1.0) < 1.0e-12
assert abs(float(endpoints[1]) - eta) < 1.0e-12
assert ad_error < 1.0e-12
assert fd_error < 1.0e-8
print({"g(0)": float(endpoints[0]), "g(1)": float(endpoints[1]), "AD_error": ad_
↪error, "FD_error": fd_error})
```

```
{'g(0)': 1.0, 'g(1)': 1e-07, 'AD_error': 0.0, 'FD_error': 3.5083047578154947e-12}
```

```
d_plot = torch.linspace(0.0, 1.0, 400, dtype=torch.float64)
fig, axes = plt.subplots(1, 2, figsize=(10, 3.5), constrained_layout=True)
axes[0].plot(d_plot, material.degradation(d_plot), label=r"$g(d)$")
axes[0].scatter([0, 1], endpoints, color="black", zorder=3)
axes[0].set(xlabel="damage d", ylabel="degradation", title="Residual stiffness␣
↪remains at d=1")
axes[0].legend()
axes[1].plot(d_plot, gp_analytic(d_plot), label=r"$g'(d)$")
axes[1].axhline(0, color="black", lw=0.7)
axes[1].set(xlabel="damage d", ylabel="derivative", title="Analytic derivative")
axes[1].legend()
fig.savefig(assets_dir() / "degradation_autograd.png", dpi=160, bbox_inches="tight
↪")
display_figure(fig, alt="Standard degradation function and its analytic derivative␣
↪over damage from zero to one.")
plt.close(fig)
```



Why this matters. PyTorch tensors make automatic differentiation available for tensor operations that remain connected to a scalar loss. A full solver derivative additionally depends on the chosen route: boundary treatment, linear/nonlinear solve, history update, detaches, clipping, and convergence behaviour all matter. The next notebook keeps that chain visible in a small original bar toy.

Exercises

Try each question before opening the hint or worked solution. Keep the original calculation as your reference.

Exercise 1: Evaluate the law, its slope, and its curvature

For

$$g(d) = (1-\eta)(1-d)^2 + \eta, \qquad \eta = 10^{-7},$$

compute $g(0)$, $g(1)$, $g(0.25)$, $g'(0.25)$, and $g''(d)$. State whether g is decreasing and whether it is convex on $0 \leq d \leq 1$.

Hint 1

Differentiate the square before substituting numbers: $g'(d) = -2(1-\eta)(1-d)$.

Worked solution 1

First,

$$g'(d) = -2(1-\eta)(1-d), \qquad g''(d) = 2(1-\eta).$$

At the endpoints,

$$g(0) = 1, \qquad g(1) = \eta = 10^{-7},$$

so the implementation retains a small residual stiffness rather than setting the broken-state factor exactly to zero. At $d = 0.25$,

$$g(0.25) = (0.9999999)(0.75)^2 + 10^{-7} = 0.56250004375,$$

and

$$g'(0.25) = -2(0.9999999)(0.75) = -1.49999985.$$

Finally, $g'' = 1.9999998 > 0$, so the law is convex. For $d < 1$, $g'(d) < 0$, hence it is decreasing over the intact-to-broken interval. These values match the retained notebook output.

Exercise 2: Read an AD/FD agreement without overclaiming

At $d = 0.25$, the notebook reports analytic and autograd derivatives of -1.49999985. Its centred finite difference uses $h = 10^{-6}$ and gives -1.4999998499964917, with absolute error 3.5083×10^{-12}.

1. Write the centred-difference formula used here.
2. Explain what this result checks.
3. Name two features of a coupled fracture route that this local check does **not** automatically differentiate through.

Hint 2

The finite difference perturbs only the scalar d in the smooth material law. Follow the computational graph only as far as the scalar function actually evaluated.

Worked solution 2

1. The numerical estimate is
$$g'_{\mathrm{FD}}(d) = \frac{g(d+h) - g(d-h)}{2h}, \qquad h = 10^{-6}.$$
2. Its agreement with the analytic and autograd values checks the local implementation of this smooth scalar degradation function at the chosen point and precision. The 3.5083×10^{-12} discrepancy is consistent with a well-resolved centred difference in **float64**.
3. It does not by itself establish derivatives through, for example, a history maximum/irreversibility update, a convergence-dependent nonlinear solve, boundary treatment, a detach, or a clamp/active set. A coupled derivative claim must identify the selected solver route and verify the entire parameter-to-loss chain.

3.4 AT2 and AT1 are normalised choices

The crack-density contribution is often written

$$\Gamma_\ell(d) = \frac{1}{c_0} \int_\Omega \left(\frac{w(d)}{\ell} + \ell |\nabla d|^2 \right) \mathrm{d}x, \qquad c_0 = 4 \int_0^1 \sqrt{w(s)}\, \mathrm{d}s.$$

With this convention:

$$\begin{aligned} &\text{AT2:} \quad w(d) = d^2, \qquad c_0 = 2, \\ &\text{AT1:} \quad w(d) = d, \qquad c_0 = \frac{8}{3}. \end{aligned}$$

The labels AT1 and AT2 are widely used, but coefficients in papers can look different because authors absorb factors into G_c, c_0, or the gradient term. Compare complete functionals, not a label alone.

3.4.1 Why the profiles look different

At a fully developed straight crack in an idealised one-dimensional setting, the AT2 profile has exponential tails,

$$d_{\mathrm{AT2}}(x) = \exp(-|x|/\ell),$$

whereas the corresponding AT1 profile is compactly supported,

$$d_{\mathrm{AT1}}(x) = \left(1 - \frac{|x|}{2\ell}\right)_+^2, \qquad (a)_+ = \max(a, 0).$$

These ideal profiles explain the left panel of the preceding figure. They do not remove the need for a mesh-convergence study in a finite domain with boundary conditions and a particular loading path.

In common one-dimensional analyses, AT1 exhibits a finite damage-initiation threshold, whereas AT2 can begin to reduce stiffness without a strictly elastic phase. That statement depends on the surrounding energy, loading, and history construction. It should not be shortened to “AT1 is always delayed” or “AT2 is always more brittle.”

3.5 Deriving the strong-form damage residual

Hold u fixed momentarily and take a virtual change δd. The damage-dependent part of the first variation is

$$\delta_d \mathcal{E}_\ell = \int_\Omega \left[g'_\eta(d)\psi^+ \, \delta d + \frac{G_c}{c_0} \left(\frac{w'(d)}{\ell} \delta d + 2\ell \nabla d \cdot \nabla(\delta d) \right) \right] \mathrm{d}x.$$

Integrating the gradient term by parts gives, away from active constraints,

$$r_d = g'_\eta(d)\psi^+ + \frac{G_c}{c_0} \left(\frac{w'(d)}{\ell} - 2\ell \Delta d \right) = 0.$$

A natural boundary condition for an unconstrained boundary is $\nabla d \cdot n = 0$. The equation has three visible influences:

1. $g'_\eta(d)\psi^+$ drives damage where the selected elastic energy is high;
2. $w'(d)/\ell$ penalises local damage; and
3. $-2\ell\Delta d$ penalises abrupt spatial variation.

The derivation also shows why a diffusion analogy is helpful but incomplete: the gradient term is diffusion-like, yet the mechanical driving energy and one-way damage history are essential.

3.6 Irreversibility is a constraint, not a plot style

For brittle damage, a common admissible set at load step n is

$$d_n(x) \geq d_{n-1}(x), \qquad 0 \leq d_n(x) \leq 1.$$

At a point where the lower bound is active, the unconstrained equation $r_d = 0$ need not hold. If only the lower bound is displayed, a local complementarity form is

$$r_d - \lambda = 0, \qquad \lambda \geq 0, \qquad d - d_{n-1} \geq 0, \qquad \lambda(d - d_{n-1}) = 0.$$

The upper bound introduces a second multiplier. Implementations may instead use a history field, a constrained solver, or another equivalent strategy under stated assumptions. A colour map that happens to look monotone is not evidence that irreversibility was imposed correctly; check the stored constraint or history data.

3.7 Width, mesh, and what must be checked

The parameter ℓ is a modelling and numerical length. It spreads the regularised crack and therefore interacts with mesh spacing h, geometry, and material calibration. A credible study reports ℓ, the mesh, and a convergence or sensitivity check. There is no universal element-count slogan that replaces this check.

Changing ℓ can change:

- the apparent width of the damaged band;
- the peak load and softening response in a finite discretisation;
- the required mesh density and nonlinear conditioning; and
- the meaning of a comparison with an observed fracture process zone.

The phase-field regularisation should not be described as a physical crack width unless that calibration has actually been established for the material and model.

3.8 Exercise: one derivative and one scale argument

1. Evaluate $g_\eta(0)$, $g_\eta(1)$, and $g'_\eta(1)$ for the law above.
2. In the damage residual, which term becomes large when ℓ is made small at fixed d and ∇d?
3. Why is it incomplete to infer mesh independence from an unchanged full-domain average of d?

Solution

1. $g_\eta(0) = 1$, $g_\eta(1) = \eta_{\text{res}}$, and $g'_\eta(1) = 0$. The derivative vanishes at the fully degraded endpoint for this quadratic law.
2. The local crack-density contribution $w'(d)/\ell$ scales inversely with ℓ. The gradient and solution fields may also change, so this is a scale observation rather than a full solution argument.
3. A spatial average can remain similar while the crack band moves, changes width, or is under-resolved. Compare fields, energies, and observables at stated refinements.

In the computational lesson, the first result is a narrower check: it compares the analytic, automatic-differentiation, and finite-difference derivatives of the selected degradation law at one stated damage value. It is evidence about that scalar operation, not about the entire coupled fracture route.

3.9 Sources for this chapter

The Ambrosio–Tortorelli regularisation idea underlies the AT terminology. For phase-field fracture derivations and comparisons, see Miehe, Welschinger, and Hofacker (2010), Pham et al. (2011), and the review by Kristensen, Niordson, and Martínez-Pañeda (2021).

CHAPTER

FOUR

FROM SOLID MECHANICS TO STAGGERED DAMAGE UPDATES

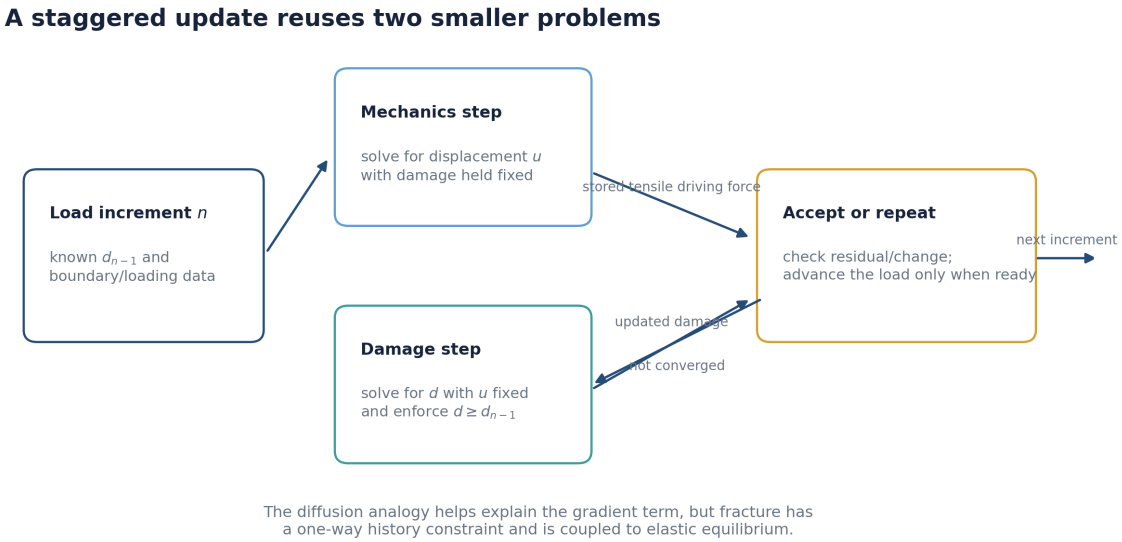


Figure 4.1: A staggered solve does not make the mechanics and damage independent; it chooses an order in which to update their coupled fields.

4.1 The mechanical subproblem

For a quasistatic small-strain body, balance of linear momentum is

$$\nabla \cdot \sigma(u, d) + b = 0 \quad \text{in } \Omega,$$

with displacement or traction boundary conditions on complementary portions of the boundary. The strain is

$$\varepsilon(u) = \frac{1}{2}\left(\nabla u + \nabla u^{\mathsf{T}}\right).$$

In a phase-field model the stress depends on damage through the selected energy split. A representative relation is

$$\sigma(u, d) = g_\eta(d)\frac{\partial \psi^+}{\partial \varepsilon} + \frac{\partial \psi^-}{\partial \varepsilon}.$$

Holding d fixed makes the mechanical problem easier to state, but it does not make it a generic linear solve: material laws, finite strain, contact, and boundary conditions may all add nonlinearity.

The weak form asks for a displacement u such that, for all admissible test fields v,

$$\int_\Omega \sigma(u, d) : \varepsilon(v)\,\mathrm{d}x = \int_\Omega b \cdot v\,\mathrm{d}x + \int_{\Gamma_t} \bar{t} \cdot v\,\mathrm{d}s.$$

This is the bridge from a continuum equation to finite elements. Shape functions turn u and v into finite vectors of degrees of freedom; an assembly or matrix-free operation then evaluates the residual.

4.2 Why the damage equation resembles diffusion

From Chapter 2, the unconstrained damage residual contains

$$\frac{w'(d)}{\ell} - 2\ell\Delta d.$$

The Laplacian has the same mathematical form as a diffusion operator. Compare the steady heat equation

$$-\nabla \cdot (k\nabla T) = q$$

with a schematic fixed-mechanics damage equation

$$-2\ell\Delta d + \frac{w'(d)}{\ell} = -\frac{c_0}{G_c} g'_\eta(d)\psi^+.$$

Both equations spread a field through a gradient penalty or conductivity-like term. The analogy helps explain why boundary conditions and element gradients matter. It is not a physical equivalence:

- the right-hand side is coupled to displacement and a selected tensile energy;
- d is constrained by a one-way damage history rather than freely diffusing; and
- the coefficients and source can change during nonlinear iteration.

Treating damage as an ordinary heat field can therefore hide the very constraint that represents irreversible fracture.

4.3 One load increment, two coupled updates

At load increment n, start with the previously accepted damage d_{n-1}. A simple staggered pattern is:

1. initialise $d_n^{(0)} = d_{n-1}$;
2. solve the mechanics problem for $u_n^{(k+1)}$ with $d_n^{(k)}$ fixed;
3. construct the specified damage-driving quantity from $u_n^{(k+1)}$;
4. solve the constrained damage problem for $d_n^{(k+1)}$;
5. test convergence of both fields and the relevant residuals; and
6. either repeat at this load level or accept the increment.

The figure above is a visual version of this algorithm. A history-field implementation may use, for example,

$$H_n(x) = \max\big(H_{n-1}(x), \psi^+(\varepsilon(u_n(x)))\big)$$

to prevent loss of the tensile driving force on unloading. That is one commonly used construction, not the definition of all phase-field models. Other approaches impose $d_n \geq d_{n-1}$ directly in the damage solve.

4.4 Staggered, monolithic, Newton, and quasi-Newton

These terms answer different algorithmic questions.

Staggered solve. Update u and d in blocks. It can be easier to implement and diagnose because each subproblem has a recognisable role. It may need several block iterations per load step and can be sensitive to coupling strength and stopping criteria.

Monolithic solve. Solve for the coupled state (u, d) together. This can better represent strong coupling, but requires a larger coupled residual, Jacobians or Jacobian actions, and careful treatment of inequality constraints.

Newton method. Linearise a residual around the current state using its Jacobian. Quadratic local convergence is a useful ideal, not an unconditional observed property of a fracture calculation.

Quasi-Newton method. Update an approximate Jacobian or inverse Jacobian from changes in residuals and states. BFGS is a well-known example for optimisation. Quasi-Newton therefore names a nonlinear solve strategy; it is neither a fracture energy nor an XFEM enrichment.

No single label supplies a convergence proof. Record the block iteration count, tolerance, residual definition, and accepted load increments.

4.5 Static and dynamic fracture

The quasistatic balance above neglects inertia. A dynamic model instead uses

$$\rho\ddot{u} - \nabla \cdot \sigma(u, d) = b.$$

Now time integration, mass treatment, wave propagation, damping choices, and time-step size affect the numerical result. A dynamic calculation may show a different crack path or timing from a quasistatic calculation even with the same stored energy, because the kinetic-energy and loading-rate terms differ.

The phrase "dynamic phase field" should therefore be accompanied by the time integrator, time step, inertia/damping assumptions, and the quantity used to enforce or approximate irreversibility.

4.6 Acceptance checks

At an accepted increment, consider at least the following checks:

- **mechanical balance:** a stated residual norm, reaction balance, or weak form defect;
- **damage update:** a stated residual or constrained-solver criterion;
- **irreversibility:** verify $d_n \geq d_{n-1}$ within the documented numerical tolerance;
- **bounds:** inspect whether d respects the intended interval;
- **energy interpretation:** report the terms and sign convention before claiming energy balance; and
- **field inspection:** view u, d, and the driving quantity, not only a scalar convergence history.

An iteration count without the residual definition is weak evidence. A decreasing residual without field inspection can still conceal a wrong boundary condition, a flipped damage convention, or an under-resolved band.

4.7 Predict before the public calculation

The integrated PhAST lesson in the next chapter uses a quasistatic staggered route. Before reading its code or retained plots, predict which two traces should change as the load is increased: one trace should describe the evolving damage outside the locked precrack, and another should record how many coupled updates were needed at each load level. Neither trace alone is a residual norm or proof of mesh convergence; use the case card and field plots as well.

4.8 Exercise: write the update before reading the code

Suppose a notched specimen is loaded in displacement control. Put the following actions in a sensible order and identify the action that enforces one-way damage:

- solve for displacement;
- accept the new load step;
- update the damage-driving quantity;
- check field/residual changes;
- solve for damage;
- initialise from the previous accepted damage.

Solution

Start by initialising from the previous accepted damage. Solve displacement at the current trial damage, update the stated driving quantity, and solve the damage subproblem subject to its irreversibility rule. Then check residuals and field changes; only after the checks pass should the load step be accepted. The lower-bound constraint $d_n \geq d_{n-1}$, or a history construction designed to enforce its effect, is the one-way component.

This ordering is the interpretation key for the later computational lesson. The lesson's stagger-iteration trace describes the configured block-update route; it should not be mistaken for a universal convergence guarantee.

4.9 Sources for this chapter

For a variational phase-field treatment and a staggered implementation route, see Miehe, Welschinger, and Hofacker (2010). A discussion of numerical formulations and length-scale effects is provided by Gerasimov and De Lorenzis (2019).

CHAPTER

FIVE

GEOMETRY, FEM, TENSORS, AND MATRIX-FREE OPERATORS

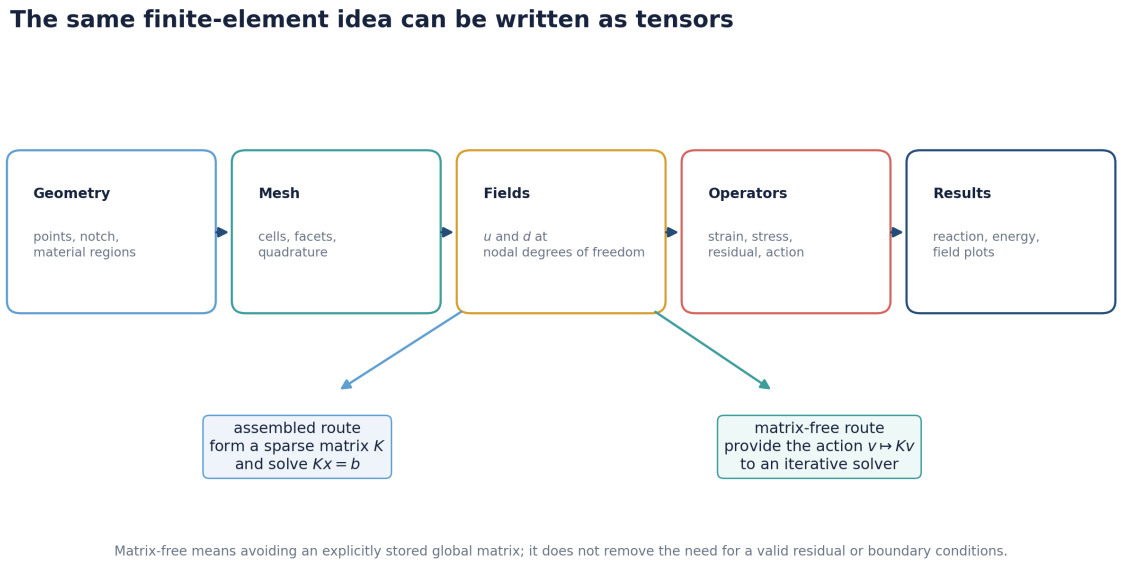


Figure 5.1: Finite-element data can be expressed as tensors without changing the need to define geometry, interpolation, quadrature, loads, and checks.

5.1 Start with a boundary-value problem

A computational fracture result should be traceable to a boundary-value problem, not merely a tensor shape. State:

- the domain Ω, notch or initial damaged region, and material regions;
- the displacement boundary Γ_u and traction boundary Γ_t;
- prescribed displacement $\bar{u}$, traction $\bar{t}$, and body force b;
- the phase-field convention and initial value d_0;
- the mesh, finite-element interpolation, quadrature, and load increments; and
- the scalar quantities and fields that will be plotted.

In standard notation,

$$u = \bar{u} \text{ on } \Gamma_u, \qquad \sigma n = \bar{t} \text{ on } \Gamma_t.$$

If a notch is represented through a prescribed initial damage field rather than a literal geometric cut, say so. These choices can produce visually similar pictures but encode different computational objects.

5.2 From cells to fields

For a two-dimensional mesh, common arrays are:

- coordinates $X \in \mathbb{R}^{N_{\text{node}} \times 2}$;
- cell connectivity $C \in \{0, \dots, N_{\text{node}} - 1\}^{N_{\text{cell}} \times n_e}$;
- nodal displacement $U \in \mathbb{R}^{N_{\text{node}} \times 2}$; and
- nodal damage $D \in \mathbb{R}^{N_{\text{node}}}$.

The cell connectivity does not contain the field values. It tells the program which rows of X, U, and D belong to each element. For an element e with local nodes a, finite-element interpolation takes the form

$$u_h(x) = \sum_{a=1}^{n_e} N_a(x) u_a, \qquad d_h(x) = \sum_{a=1}^{n_e} N_a(x) d_a.$$

Differentiating the shape functions maps nodal displacement to strain. In matrix notation at one quadrature point,

$$\varepsilon_h = B_e u_e.$$

The element residual is computed by integrating the weak form. A schematic mechanical contribution is

$$r_e(u, d) = \int_{\Omega_e} B_e^{\mathsf{T}} \sigma(u_h, d_h) \, \mathrm{d}x - f_e^{\text{ext}}.$$

Assembly scatters each local residual into the appropriate entries of the global residual. The same operation can be expressed with index operations, scatter-adds, or framework-specific sparse primitives.

5.3 Tensor shapes are part of the model contract

Write expected shapes next to a tensor operation. For example, with batched cells and quadrature points, an implementation might use:

- cell coordinates: $(N_{\text{cell}}, n_e, 2)$;
- shape-function gradients: $(N_{\text{cell}}, N_q, n_e, 2)$;
- gathered displacement: $(N_{\text{cell}}, n_e, 2)$;
- strain or stress: $(N_{\text{cell}}, N_q, n_\varepsilon)$; and
- quadrature weights: (N_{cell}, N_q).

The exact ordering is a convention. A transpose that preserves the total number of entries can still mix components, cells, and quadrature points. Use small assertions and one element-level hand calculation before relying on a large field plot.

Indexing check

Select one cell and one quadrature point. Write the gathered nodal values, shape gradients, strain, stress, and residual contribution on paper. If those five objects agree with the tensor code, then vectorising over cells has a clear reference calculation.

5.4 Assembled and matrix-free routes

For a linearised subproblem, an assembled finite-element method forms a global matrix

$$K = \sum_e A_e^\mathsf{T} K_e A_e$$

and solves $Kx = b$ or a linearised residual equation. A_e symbolically represents the local-to-global map.

A matrix-free method instead implements the action

$$v \longmapsto Kv$$

by gathering v to elements, applying local operations, and scattering contributions back. An iterative Krylov solver can use this action without storing every entry of K.

Matrix-free does **not** mean "no matrix mathematics," "no memory cost," or "automatically faster." It moves attention to the correctness and efficiency of residual/Jacobian actions, preconditioning, data movement, and stopping criteria. Compare methods at matched physics, mesh, tolerance, and reported observable.

5.5 Static and dynamic arrays

In a quasistatic calculation, the state often consists primarily of U and D. A dynamic calculation adds at least velocity and acceleration or their time-integration equivalents. A schematic time step must then track

$$(U_n, V_n, A_n, D_n) \longrightarrow (U_{n+1}, V_{n+1}, A_{n+1}, D_{n+1}).$$

The extra arrays are not bookkeeping only: their update rule expresses inertia and affects the computed path. Do not label a code dynamic merely because it loops over a sequence of prescribed displacements.

5.6 Where automatic differentiation fits

PyTorch provides tensors and a reverse-mode automatic-differentiation system; it is not, by itself, a fracture finite-element formulation. The user or a library still defines mesh data, interpolation, constitutive response, residuals, boundary conditions, and solver behaviour.

JAX likewise provides array transformations such as differentiation, compilation, and vectorisation. JAX-FEM is an additional finite-element library built on that ecosystem and includes worked mechanics examples. Neither ecosystem is universally superior for this course: a suitable choice depends on the existing solver, device support, reproducibility needs, and whether the relevant operations are differentiable in the intended computation.

See the official PyTorch autograd documentation, the JAX automatic-differentiation guide, and the JAX-FEM quick-start.

5.7 Inspecting results: four views, one interpretation

For a tiny evolving-fracture run, inspect at least:

1. the geometry, mesh, and loading direction;
2. displacement or deformed configuration with a stated scale factor;
3. damage field with a labelled colour range and the $d = 0/d = 1$ convention; and
4. a scalar observation such as reaction force, energy term, or residual.

5.8 Computational lesson: follow a public PhAST calculation

The lesson below is the book's small public PhAST example. It defines a rectangular T3 mesh, labelled boundaries, a locked centreline precrack, and symmetric displacement loading before showing the damage fields, response trace, and stagger-iteration trace. Read the case card before interpreting the plots. The route is quasistatic and uses an assembled sparse-direct mechanics solve; it is neither a dynamic calculation nor a demonstration that no matrices are constructed.

5.8.1 Run and interpret a small PhAST fracture calculation

Learning objective. Trace the geometry, T3 mesh, boundary conditions, solver route, and retained evidence for the actual public PhAST AT2 calculation. Separate what this one quasi-static result establishes from claims about physical dynamics or a sharp crack front.

Predict first. Before reading the receipt, predict whether the mesh contains more nodes or elements, and whether a positive change in damage outside the locked notch alone proves a resolved propagating crack front.

Recorded computation: 76.301 seconds on the reference local CPU after setup. This is not a fresh Colab timing.

Read the code and its recorded outputs in order. To change inputs and run Python, download a notebook and use the course environment. The static book does not execute these cells in the browser.

Download practice notebook · Download with worked solutions · Environment setup

This is an actual **public PhAST** AT2 phase-field solve, run on a CPU with a structured triangular mesh. It makes a rectangle, identifies its boundaries, applies symmetric tension, locks a centreline precrack, and advances sixty **quasi-static load increments**. It is not a physical-time dynamic simulation and it does not use XFEM.

The execution pin is `CEMS-Lab/PhAST@f6324f899f0701769810be117f27f1208f7a582e` (version 0.16.2). The tested local route is source-first import: `PYTHONPATH=<public-PhAST>/src python`. The course bundle prefers `vendor/PhAST/src` and verifies its manifest if Git metadata is not present. `python -m pip install ./vendor/PhAST` is a preparatory setup option, not part of the recorded notebook timing. The next cell verifies that the public pin, rather than a neighbouring private checkout, was imported.

What to look for. `d=0` means intact-like and `d=1` means fully broken. The locked centreline is only the initial crack. The calculation must show a positive increment of damage outside that set as load increases; otherwise it is a setup check, not the promised evolving-damage result.

```
from pathlib import Path
import sys

def locate_course_root():
    for base in (Path.cwd(), *Path.cwd().parents, Path(__file__).resolve().parent␣
↪if "__file__" in globals() else Path.cwd()):
        direct = base / "teaching" / "ukacm_autumn_school_2026"
        if (direct / "notebooks" / "day2_helpers").is_dir():
            return direct
        if (base / "notebooks" / "day2_helpers").is_dir():
            return base
    raise FileNotFoundError("Could not find teaching/ukacm_autumn_school_2026.")

COURSE_ROOT = locate_course_root()
if str(COURSE_ROOT / "notebooks") not in sys.path:
    sys.path.insert(0, str(COURSE_ROOT / "notebooks"))

import matplotlib.pyplot as plt
import numpy as np
import torch
from io import BytesIO
from IPython.display import Image, display

from day2_helpers import assets_dir

torch.set_default_dtype(torch.float64)
```

(continues on next page)

(continued from previous page)

```
def display_figure(figure, dpi=150, alt="Rendered teaching figure"):
    """Embed a PNG even when nbclient uses a non-interactive Agg backend."""
    buffer = BytesIO()
    figure.savefig(buffer, format="png", dpi=dpi, bbox_inches="tight")
    display(Image(data=buffer.getvalue(), alt=alt))

print({"course_layout": "teaching/ukacm_autumn_school_2026 located", "torch":␣
↪torch.__version__, "device": "cpu", "dtype": str(torch.get_default_dtype())})
```

```
{'course_layout': 'teaching/ukacm_autumn_school_2026 located', 'torch': '2.8.0',
↪'device': 'cpu', 'dtype': 'torch.float64'}
```

```
from day2_helpers import import_public_phast, load_forward_config, run_notched_
↪tension

config = load_forward_config()
phast_info = import_public_phast(config["public_source"]["revision"])
print(phast_info)
print({k: config[k] for k in ("geometry", "mesh", "material", "loading", "solver",
↪"limits")})
```

```
{'source': 'vendor/PhAST/src (manifest-verified when vendored)', 'module_file':
↪'phast/__init__.py', 'revision': 'f6324f899f0701769810be117f27f1208f7a582e',
↪'version': '0.16.2', 'import_route': 'PYTHONPATH=<public-PhAST>/src',
↪'verification': 'COURSE_SOURCE_MANIFEST.json hashes verified'}
{'geometry': {'width': 4.0, 'height': 2.0, 'notch_length': 0.8, 'description': 'A␣
↪rectangular specimen with a centreline phase-field Dirichlet precrack from x=0␣
↪to x=notch_length.'}, 'mesh': {'nx': 128, 'ny': 64, 'element_type': 'T3', 'dtype
↪': 'float64'}, 'material': {'E': 1.0, 'nu': 0.3, 'Gc': 0.0005, 'l0': 0.15, 'rho
↪': 1.0, 'energy_split': 'isotropic', 'pf_model': 'AT2', 'eta_residual': 1e-07},
↪'loading': {'total_symmetric_vertical_displacement': 0.04, 'n_load_steps': 60,
↪'first_load_factor': 0.0125, 'last_load_factor': 1.0}, 'solver': {'solver_type':
↪'quasi_static', 'mechanics_backend': 'scipy', 'damage_preconditioner': 'jacobi',
↪'use_multigrid': False, 'static_tol': 1e-08, 'static_max_iter': 1000, 'damage_tol
↪': 1e-08, 'damage_max_iter': 1000, 'stagger_tol': 1e-05, 'max_stagger': 200,
↪'fail_on_mechanics_nonconvergence': True, 'fail_on_stagger_nonconvergence': True}
↪, 'limits': {'solver_seconds_target_low': 60, 'solver_seconds_target_high': 120,
↪'solver_seconds_hard': 300, 'notebook_seconds_hard': 300}}
```

Geometry, mesh, labelled boundaries, material, and constraints

The next cell is the same transparent setup used by the runner. It first generates the T3 mesh in memory, then saves and reloads a small prepared **tensor-mesh** file. This is a recovery/import route for the course specimen—not a claim that an external mesh file with physical groups was used. `identify_boundaries()` makes the `left`, `right`, `top`, and `bottom` node labels; the explicit centreline mask labels the precrack.

```
from day2_helpers.course_tools import _structured_triangles
from phast.core.mesh import FEMMesh
from phast.physics.boundary_conditions import symmetric_tension_bcs
from phast.physics.material import Material

geometry, mesh_cfg, material_cfg, loading = (
    config["geometry"], config["mesh"], config["material"], config["loading"]
)
width, height = geometry["width"], geometry["height"]
x = torch.linspace(0.0, width, mesh_cfg["nx"] + 1, dtype=torch.float64)
y = torch.linspace(0.0, height, mesh_cfg["ny"] + 1, dtype=torch.float64)
xx, yy = torch.meshgrid(x, y, indexing="xy")
```

(continues on next page)

(continued from previous page)

```
nodes_preview = torch.stack((xx.reshape(-1), yy.reshape(-1)), dim=1)
elements_preview = _structured_triangles(mesh_cfg["nx"], mesh_cfg["ny"])
mesh_preview = FEMMesh.from_tensors(nodes_preview, elements_preview, device="cpu",␣
↪dtype=torch.float64)
mesh_preview.identify_boundaries()

bcs_preview = symmetric_tension_bcs(mesh_preview, disp=loading["total_symmetric_
↪vertical_displacement"])
precrack_mask = ((nodes_preview[:, 1] - height / 2).abs() < 1.0e-12) & (nodes_
↪preview[:, 0] <= geometry["notch_length"] + 1.0e-12)
precrack_nodes = torch.where(precrack_mask)[0]
bcs_preview.add_pf_dirichlet(precrack_nodes, value=1.0)
material_preview = Material(
    E=material_cfg["E"], nu=material_cfg["nu"], Gc=material_cfg["Gc"], l0=material_
↪cfg["l0"],
    rho=material_cfg["rho"], eta_residual=material_cfg["eta_residual"],
    energy_split=material_cfg["energy_split"], pf_model=material_cfg["pf_model"],
)

prepared_mesh = assets_dir() / "tiny_notched_tension_prepared_mesh.npz"
np.savez_compressed(
    prepared_mesh,
    nodes=nodes_preview.numpy(), elements=elements_preview.numpy(), precrack_
↪nodes=precrack_nodes.numpy(),
    **{name: index.numpy() for name, index in mesh_preview.node_sets.items()},
)
imported = np.load(prepared_mesh)
assert np.array_equal(imported["elements"], elements_preview.numpy())
print({
    "geometry": f"{width} × {height} rectangle; centreline notch length {geometry[
↪'notch_length']}",
    "mesh": f"{mesh_preview.n_nodes} nodes / {mesh_preview.n_elems} T3 elements",
    "boundary_labels": sorted(mesh_preview.node_sets),
    "precrack_nodes": len(precrack_nodes),
    "prepared_tensor_mesh": "assets/day2_forward/tiny_notched_tension_prepared_
↪mesh.npz",
    "material": {key: material_cfg[key] for key in ("E", "nu", "Gc", "l0", "pf_
↪model")},
    "BC": "x fixed on exterior; top/bottom symmetric vertical displacement; d=1 on␣
↪precrack",
})
```

```
[FEMMesh.from_tensors] 8385 nodes, 16384 T3 elements, h_min=1.830583e-02
```

```
{'geometry': '4.0 × 2.0 rectangle; centreline notch length 0.8', 'mesh': '8385␣
↪nodes / 16384 T3 elements', 'boundary_labels': ['bottom', 'left', 'right', 'top
↪'], 'precrack_nodes': 26, 'prepared_tensor_mesh': 'assets/day2_forward/tiny_
↪notched_tension_prepared_mesh.npz', 'material': {'E': 1.0, 'nu': 0.3, 'Gc': 0.
↪0005, 'l0': 0.15, 'pf_model': 'AT2'}, 'BC': 'x fixed on exterior; top/bottom␣
↪symmetric vertical displacement; d=1 on precrack'}
```

Route and representation

The selected mechanics route is `solver_type="quasi_static"` with the SciPy sparse-direct backend. It performs a staggered update: mechanics → tensile/history update → phase-field damage → repeat to the configured tolerance. The public implementation contains both assembled sparse and matrix-free element-operator paths; this notebook demonstrates the assembled sparse-direct mechanics route, so it does not claim that no matrices are constructed.

A quasi-Newton/Newton method would be a nonlinear **solution algorithm**, while XFEM would be a crack **representation**. The present calculation instead uses a regularised phase-field formulation on T3 elements.

```
# The helper writes a small JSON/NPZ receipt under assets/day2_forward/.
# It also has a 300-second limit recorded in the configuration;
# the external notebook executor enforces the process-level timeout.
result = run_notched_tension(config)
summary = result["summary"]
summary
```

```
[FEMMesh.from_tensors] 8385 nodes, 16384 T3 elements, h_min=1.830583e-02
```

```
[StaggeredSolver] Assembling solver (quasi_static)...
```

```
[FEMOperators] Initializing (isotropic split, AT2)...
```

```
[FEMOperators] dt_CFL = 1.577764e-02 (c_p=1.16)
```

```
[PhaseFieldDamageSolver] Initializing CG solver (model=AT2, tol=1.0e-08, max_
↪iter=1000, bounds=post_clamp)...
```

```
[PhaseFieldDamageSolver] CG device: cpu, dtype: float64
```

```
[PhaseFieldDamageSolver] Ready (preconditioner=jacobi).
```

```
[StaggeredSolver] QuasiStaticSolver ready
```

```
[StaggeredSolver] State allocated: 8385 nodes, 16384 elements, device=cpu,␣
↪dtype=torch.float64
```

```
[PhAST route]
```

```
  device=cpu; dtype=torch.float64; elements=T3
```

```
  mechanics=QuasiStaticSolver; solver_type=quasi_static; time_integrator=central_
↪difference; backend_requested=scipy; backend_resolved=at first solve
```

```
  fracture=AT2; energy_split=isotropic; history_update=hard_max; damage_
↪update=classical; bounds=post_clamp; preconditioner=jacobi
```

```
[QuasiStaticSolver] mechanics backend: scipy (SciPy SuperLU sparse-direct LU);␣
↪tangent=isotropic linear tangent; free_dofs=16128
```

```
{'nodes': 8385,
 'elements': 16384,
 'load_steps': 60,
 'dtype': 'float64',
 'setup_seconds': 0.012829625004087575,
 'solve_seconds': 70.07502354199823,
```

(continues on next page)

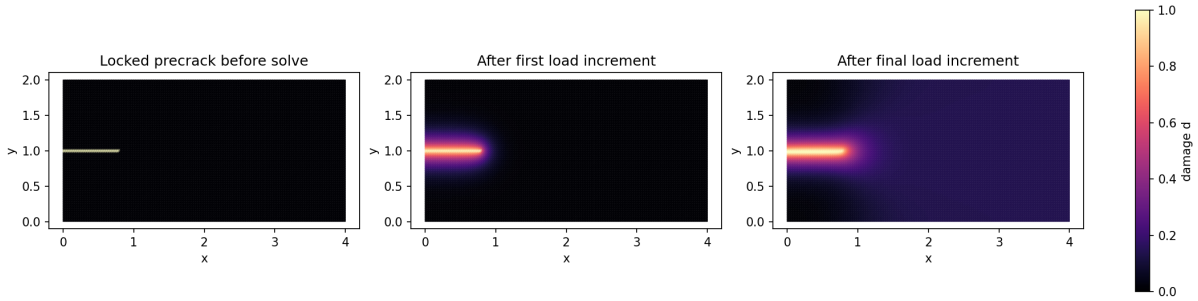
(continued from previous page)

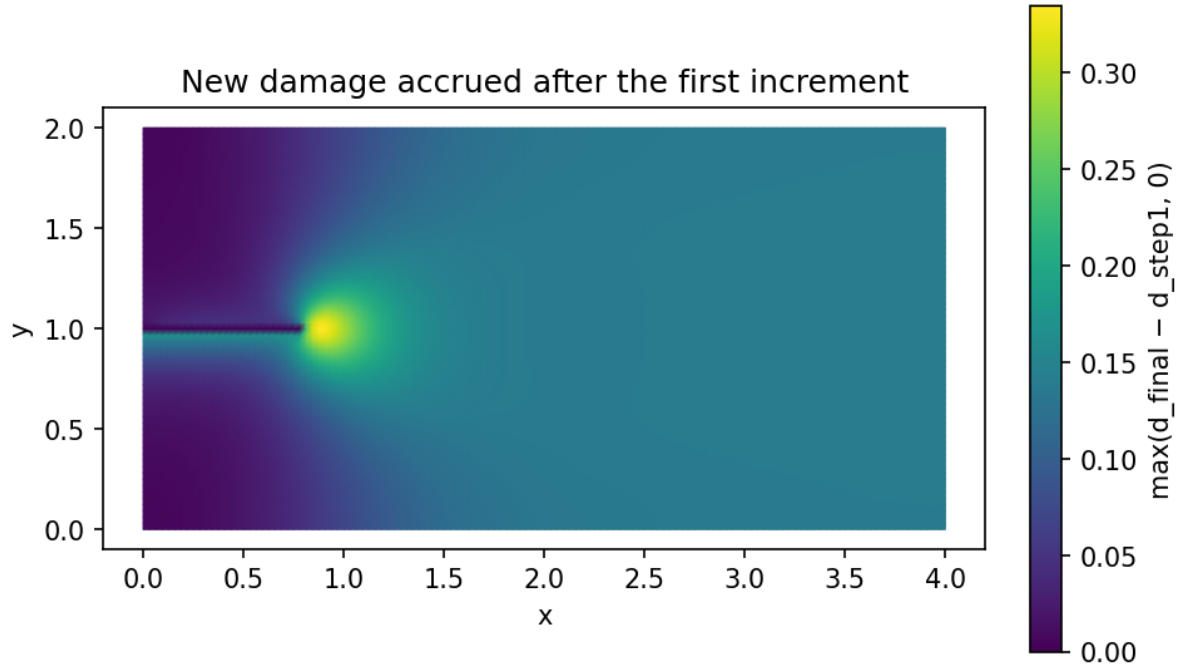
```
 'damage_outside_initial_sum': 246.03417370951848,
 'damage_outside_final_sum': 1213.493648486593,
 'incremental_damage_outside_sum': 967.4594747770745,
 'incremental_damage_outside_max': 0.33508128500651757,
 'final_damage_outside_max': 0.9827202060098374,
 'all_steps_returned_with_failure_flags_enabled': True,
 'warnings': []}
```

```
# The checks describe the promised evidence, not an aesthetic judgement.
assert summary["solve_seconds"] < config["limits"]["solver_seconds_hard"]
assert summary["incremental_damage_outside_sum"] > 1.0
assert summary["incremental_damage_outside_max"] > 1.0e-3
assert not summary["warnings"], summary["warnings"]
print("Evolving-damage assertions passed.")
```

```
Evolving-damage assertions passed.
```

```
import matplotlib.tri as mtri

nodes = result["nodes"].detach().cpu().numpy()
triangles = result["elements"].detach().cpu().numpy()
tri = mtri.Triangulation(nodes[:, 0], nodes[:, 1], triangles)
seeded = result["seeded_damage"].detach().cpu().numpy()
first = result["first_damage"].detach().cpu().numpy()
final = result["final_damage"].detach().cpu().numpy()
increment = result["incremental_damage"].detach().cpu().numpy()

fig, axes = plt.subplots(1, 3, figsize=(14, 3.5), constrained_layout=True)
for ax, field, title in zip(
    axes,
    (seeded, first, final),
    ("Locked precrack before solve", "After first load increment", "After final␣
↪load increment"),
):
    image = ax.tripcolor(tri, field, shading="gouraud", vmin=0, vmax=1, cmap="magma
↪")
    ax.triplot(tri, color="white", lw=0.06, alpha=0.20)
    ax.set(title=title, xlabel="x", ylabel="y", aspect="equal")
fig.colorbar(image, ax=axes, label="damage d")
fig.savefig(assets_dir() / "tiny_notched_tension_damage.png", dpi=160, bbox_inches=
↪"tight")
display_figure(fig, alt="Three PhAST damage fields: locked precrack, first load␣
↪increment, and final diffuse damage field.")
plt.close(fig)

fig, ax = plt.subplots(figsize=(6.4, 3.6), constrained_layout=True)
ax.tripcolor(tri, increment, shading="gouraud", cmap="viridis")
ax.set(title="New damage accrued after the first increment", xlabel="x", ylabel="y
↪", aspect="equal")
fig.colorbar(ax.collections[0], ax=ax, label="max(d_final - d_step1, 0)")
fig.savefig(assets_dir() / "tiny_notched_tension_increment.png", dpi=160, bbox_
↪inches="tight")
display_figure(fig, alt="Map of incremental damage accumulated after the first␣
↪load increment.")
plt.close(fig)
```



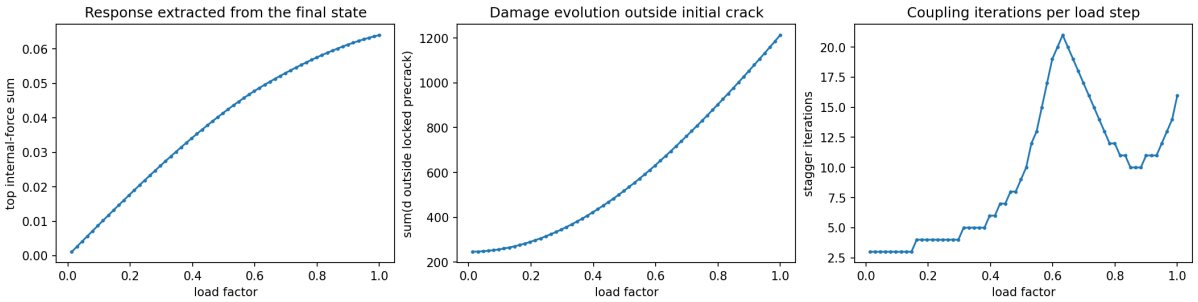



```
trace = result["trace"]
load = np.array([row["load_factor"] for row in trace])
reaction = np.array([row["reaction_top_y"] for row in trace])
outside_sum = np.array([row["damage_outside_sum"] for row in trace])
stagger = np.array([row["stagger_iterations"] for row in trace])

fig, axes = plt.subplots(1, 3, figsize=(14, 3.5), constrained_layout=True)
axes[0].plot(load, reaction, marker="o", ms=2)
axes[0].set(xlabel="load factor", ylabel="top internal-force sum", title="Response␣
↪extracted from the final state")
axes[1].plot(load, outside_sum, marker="o", ms=2)
axes[1].set(xlabel="load factor", ylabel="sum(d outside locked precrack)", title=
↪"Damage evolution outside initial crack")
axes[2].plot(load, stagger, marker="o", ms=2)
axes[2].set(xlabel="load factor", ylabel="stagger iterations", title="Coupling␣
↪iterations per load step")
fig.savefig(assets_dir() / "tiny_notched_tension_response.png", dpi=160, bbox_
↪inches="tight")
display_figure(fig, alt="PhAST load response, damage-outside-precrack trace, and␣
↪stagger iteration count.")
plt.close(fig)

print({
    "mesh": f"{summary['nodes']} nodes / {summary['elements']} T3 elements",
    "setup_seconds": round(summary["setup_seconds"], 2),
    "solve_seconds": round(summary["solve_seconds"], 2),
    "incremental_damage_outside_sum": round(summary["incremental_damage_outside_sum
```

(continues on next page)

(continued from previous page)

```
↪"], 4),
    "incremental_damage_outside_max": round(summary["incremental_damage_outside_max
↪"], 4),
})
```




```
{'mesh': '8385 nodes / 16384 T3 elements', 'setup_seconds': 0.01, 'solve_seconds':␣
↪70.08, 'incremental_damage_outside_sum': 967.4595, 'incremental_damage_outside_
↪max': 0.3351}
```

Interpret, then change one input

The reaction plotted here is the sum of the internal force at the top prescribed-displacement nodes; it is a compact response diagnostic, not a calibrated experimental force. The damage fields and the increase outside the locked precrack are the key evidence that the computation evolved.

Try changing only `Gc`, `l0`, the total displacement, or the mesh density in `configs/day2_forward/tiny_notched_tension.json`, then rerun. State both the observed change and a limitation: this is a small plane problem with a hand-made mesh, a selected energy split, and no mesh-convergence study. Do not compare its wall time to a dynamic impact example or to an unmeasured Colab run.

Exercises

Try each question before opening the hint or worked solution. Keep the original calculation as your reference.

Exercise 1: Count the mesh and the locked precrack

The configuration uses a 4.0×2.0 rectangle, $n_x = 128$ and $n_y = 64$ rectangular cells, and splits each cell into two T3 triangles. The centreline precrack threshold has length 0.8.

1. Derive the number of nodes and T3 elements.
2. Compute the horizontal node spacing, identify the last grid node satisfying $x \leq 0.8$, and count the locked centreline nodes.
3. Match your counts to the geometry/mesh cell in the notebook and explain the discretisation effect at the nominal notch endpoint.

Hint 1

A structured grid with n_x cells has $n_x + 1$ nodes in that direction. A T3 split contributes two triangles per rectangle. For the notch, first find $\Delta x = 4/128$.

Worked solution 1

1. The nodal grid has

$$N = (128 + 1)(64 + 1) = 129 \times 65 = 8385.$$

There are 128×64 rectangles and two T3 elements per rectangle, so

$$N_e = 2(128)(64) = 16384.$$

2. The horizontal spacing is $\Delta x = 4/128 = 0.03125$. The nominal threshold spans $0.8/0.03125 = 25.6$ spacings, so there is no grid node exactly at $x = 0.8$. The largest admissible integer index is $\lfloor 25.6 \rfloor = 25$, whose coordinate is $25(0.03125) = 0.78125$. Nodes with indices $0, \ldots, 25$ are locked, giving $25+1 = 26$ centreline precrack nodes.
3. The retained setup reports **8385 nodes / 16384 T3 elements** and **precrack_nodes: 26**, so the hand count agrees. This is a discretisation effect: the configuration specifies a threshold at 0.8, while the nodal Dirichlet mask actually ends at the last represented centreline node not exceeding it, $x = 0.78125$. This is an in-memory prepared tensor-mesh route; it is not evidence of an external physical-group mesh import.

Exercise 2: State exactly what the forward receipt supports

The retained result reports **solver_type=quasi_static**, the SciPy SuperLU sparse-direct mechanics backend, 60 load increments, **incremental_damage_outside_sum=967.4595**, **incremental_damage_outside_max=0.3351**, and **final_damage_outside_max=0.9827**. It also prints a central-difference integrator label.

Classify each statement as supported or unsupported by this receipt, and justify it in one or two sentences:

1. "This is an actual public PhAST phase-field calculation with new damage outside the initially locked precrack."
2. "This is a physical-time dynamic impact simulation because the log mentions central difference."
3. "This verifies a sharp, resolved crack-front propagation result."
4. "This notebook demonstrates a matrix-free mechanics route."

Hint 2

Use the configured solver type and mechanics backend to classify the route. Then distinguish a positive damage-change diagnostic from a claim about a sharp front or physical time.

Worked solution 2

1. **Supported.** The notebook runs the pinned public PhAST AT2 route with 60 quasi-static load increments, and both incremental-damage diagnostics are strictly positive outside the initially locked precrack. That is evidence of evolving diffuse damage.
2. **Unsupported.** The selected route is explicitly quasi-static; the lesson does not treat its load increments as physical time. A label in a solver log does not change the modelling classification into a verified dynamic impact calculation.
3. **Unsupported.** The final field is a diffuse-damage result. The receipt deliberately does not claim a sharp front, mesh-converged crack path, branching result, or dynamic fracture validation.
4. **Unsupported for this notebook.** Public PhAST contains assembled and matrix-free element-operator paths, but this run selects the SciPy sparse-direct assembled mechanics backend. The correct statement is that the *implementation offers* both routes while this lesson demonstrates the assembled sparse-direct one.

A quasi-Newton/Newton choice would describe a nonlinear solution algorithm; XFEM would describe a discontinuity representation. Neither label should be substituted for the phase-field formulation used here.

Interpret each image using the case definition; do not crop away a boundary, notch, colourbar, or load increment that makes the result intelligible. The lesson's retained result card belongs to its recorded environment. If you download it for execution, record a new result card for your own environment.

5.9 Exercise: identify a silent shape error

A tensor containing displacement at cell nodes has shape $(N_{\text{cell}}, n_e, 2)$. A tensor of shape-function gradients has shape $(N_{\text{cell}}, N_q, n_e, 2)$. Which new axes must be aligned or introduced before forming a displacement gradient at every quadrature point, and what physical quantity should you recover for an affine displacement field?

Solution

The nodal displacement needs a quadrature axis, for example by treating it as $(N_{\text{cell}}, 1, n_e, 2)$, so that it aligns with the N_q axis of the shape gradients. Contract the local-node axis and preserve the displacement component and spatial-derivative axes according to the chosen convention. For an affine displacement field, the displacement gradient and its symmetric strain should be constant across all elements and quadrature points (up to round-off). This is an excellent first unit test. The PhAST lesson makes the mesh, nodal fields, and boundary labels visible; this one-element affine test remains the right additional check before treating a vectorised tensor route as trustworthy.

5.10 Take-away

Tensors make data movement and differentiation explicit. They do not replace the finite-element questions: what is interpolated, where it is integrated, how boundary conditions are enforced, and how the resulting field is checked.

CHAPTER

SIX

DIFFERENTIATION AND AN ILLUSTRATIVE INVERSE PROBLEM

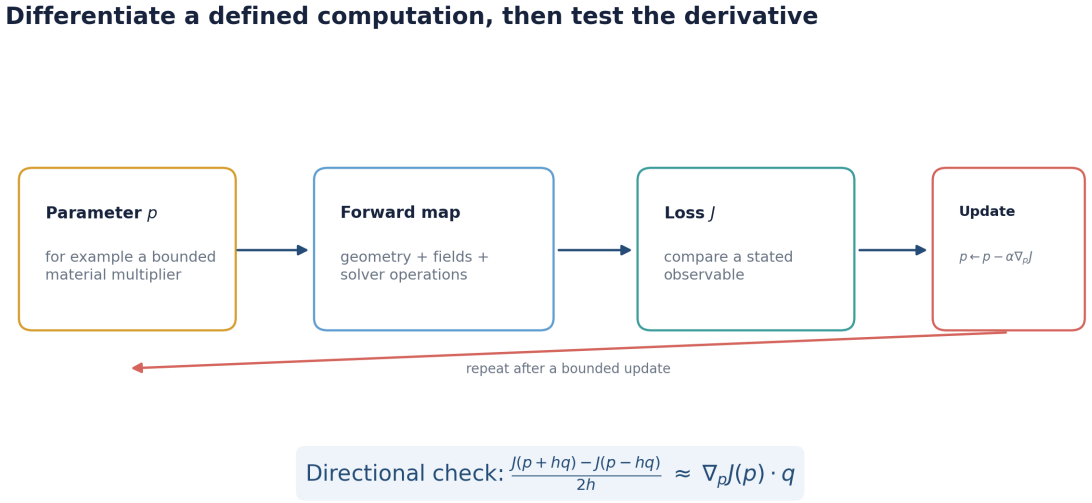


Figure 6.1: Automatic differentiation follows the stated computational graph. A derivative check asks whether that graph implements the derivative you intended.

6.1 Define the forward map before differentiating it

Let p denote one scalar parameter, such as a bounded multiplier of a material field. A forward computation maps it to a state and an observable:

$$p \longmapsto (u(p), d(p)) \longmapsto y(p).$$

With a target observation $y^\star$, an illustrative least-squares objective is

$$J(p) = \frac{1}{2}\|y(p) - y^\star\|_2^2.$$

The adjective *illustrative* matters. A small objective can teach the chain rule, tensor shapes, and optimisation bookkeeping. It cannot establish that a material parameter is identifiable from one observation.

Before calling a gradient routine, write down:

- the parameter and its units or nondimensionalisation;
- the geometry, loads, and boundary conditions held fixed;
- the observable y and the target $y^\star$;
- every operation from p to J; and

- what happens when a nonlinear solver fails, reaches a tolerance, clips a field, or changes its iteration count.

These are part of the mathematical map, not merely implementation details.

6.2 Three ways to obtain a sensitivity

6.2.1 Finite differences

A directional central difference uses a chosen direction q and step h:

$$D_h J(p; q) = \frac{J(p + hq) - J(p - hq)}{2h}.$$

It is simple and independent of the autodiff implementation, which makes it valuable as a check. It also has a step-size trade-off: a large h includes nonlinear curvature, while a very small h can suffer cancellation and solver tolerance effects.

6.2.2 Unrolled automatic differentiation

If a program executes a fixed differentiable sequence of operations, reverse mode can differentiate the sequence actually run. For example, unrolling K iterations gives

$$x_0 \to x_1 \to \cdots \to x_K \to J.$$

The derivative is of that truncated algorithm. If K changes by parameter value or a non-differentiable branch is taken, the interpretation must mention it. Memory cost can also increase when many states are retained for a reverse pass.

6.2.3 Implicit differentiation

Suppose a converged state is defined by a residual

$$R(x, p) = 0.$$

Under appropriate differentiability and nonsingularity assumptions,

$$\frac{\mathrm{d}x}{\mathrm{d}p} = -\left(\frac{\partial R}{\partial x}\right)^{-1} \frac{\partial R}{\partial p}.$$

In practice, an adjoint formulation often avoids forming this inverse explicitly. This method differentiates a local equation for a converged state, not necessarily the iterations used to obtain it. Inequality constraints, active-set changes, damage irreversibility, and non-smooth constitutive choices require particular care.

6.3 Follow the gradient one update at a time

The worked section below expands the chain rule into local derivatives, backward-propagated sensitivities and the sum of all uses of a shared parameter. Work through the three scalar updates before interpreting an autograd call as a derivative of a fracture calculation.

6.3.1 Backpropagation through a stepped update

This original, small example makes the bookkeeping behind reverse-mode automatic differentiation visible. It is deliberately an algebraic recurrence, not a fracture calculation. For a complementary visual introduction to reverse-mode differentiation in physics, see the Physics-Based Deep Learning discussion of differentiable physics. That link is a pedagogical reference only: no external code, figure, or derivation is reproduced here.

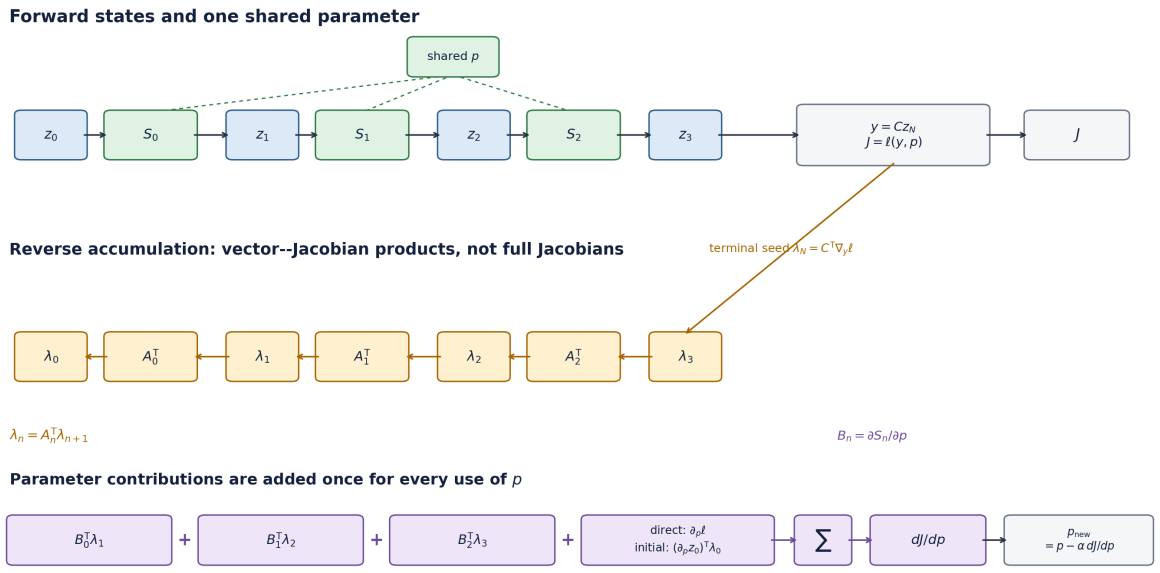


Figure 6.2: One parameter can influence a final loss at every update. Reverse mode passes a cotangent backwards and adds one parameter contribution per use of that parameter.

Start from a sequence of update maps

Let a state evolve through N discrete updates,

$$z_{n+1} = S_n(z_n, p), \qquad n = 0, \dots, N-1,$$

where the same parameter vector p may appear in every S_n. A final field or state is observed by $y = Cz_N$, and a scalar loss is formed as

$$J = \ell(y, p) = \ell(Cz_N, p).$$

Here C is assumed fixed with respect to p. If an observation operator also depends on p, its derivative belongs in the direct parameter dependence of the final observable/loss.

Define the local state and parameter derivatives

$$A_n = \frac{\partial S_n}{\partial z_n}, \qquad B_n = \frac{\partial S_n}{\partial p}.$$

For a scalar loss, the terminal adjoint is the vector

$$\lambda_N = C^\mathsf{T} \nabla_y \ell,$$

and reverse accumulation is

$$\lambda_n = A_n^\mathsf{T} \lambda_{n+1} \qquad (n = N-1, \dots, 0).$$

The resulting **total derivative** is

$$\frac{dJ}{dp} = \frac{\partial \ell}{\partial p} + \sum_{n=0}^{N-1} B_n^\mathsf{T} \lambda_{n+1} + \left(\frac{\partial z_0}{\partial p}\right)^\mathsf{T} \lambda_0.$$

This is dJ/dp, the gradient of the final scalar observable/loss with respect to the input parameter p. An optimisation algorithm uses that gradient in a *separate* choice such as

$$p_{\text{new}} = p - \alpha \frac{dJ}{dp},$$

where the learning rate α and any line search, regularisation, or constraint handling belong to the optimisation method rather than to reverse accumulation itself.

If z_N is a field, λ_N has one entry per field degree of freedom. For a scalar loss, the loss fixes the seed. For a vector-valued output, choose a seed v to obtain a vector–Jacobian product $A_n^{\mathsf{T}} v$. Neither case requires forming a full Jacobian matrix. This is why a scalar `loss.backward()` can be economical even when the state is large.

A three-step scalar calculation that can be checked by hand

Use the explicit recurrence

$$x_{n+1} = (1 - \Delta t\, p) x_n + \Delta t\, f, \qquad J = \frac{1}{2}(x_3 - y_\star)^2.$$

It is a relaxation/load **analogue only**, not PhAST, phase-field fracture, or a claim about a finite-element time integrator. The parameter p appears at every step, so it provides a clean miniature of the shared-parameter sum. For this scalar homogeneous relaxation, choosing $p > 0$ and $0 < \Delta t\, p < 1$ makes $0 < 1 - \Delta t\, p < 1$. That is a property of this one recurrence, not a universal CFL or solver-stability rule.

Take

$$\Delta t = 0.2, \quad p = 0.8, \quad f = 1.5, \quad x_0 = 0.4, \quad y_\star = 0.9.$$

Then $A_n = \partial x_{n+1}/\partial x_n = 1 - \Delta t p = 0.84$ and $B_n = \partial x_{n+1}/\partial p = -\Delta t x_n$. The forward states and local parameter derivatives are:

step n	x_n	x_{n+1}	$B_n = -\Delta t x_n$
0	0.400000	0.636000	-0.0800000
1	0.636000	0.834240	-0.1272000
2	0.834240	1.000762	-0.1668480

The terminal seed is $\lambda_3 = x_3 - y_\star = 0.1007616$. Therefore

$$\lambda_2 = 0.084639744, \qquad \lambda_1 = 0.07109738496, \qquad \lambda_0 = 0.0597218033664.$$

The corresponding reverse contributions are:

step n	λ_{n+1}	$B_n \lambda_{n+1}$
0	0.07109738	-0.005687791
1	0.08463974	-0.010766175
2	0.10076160	-0.016811871

The displayed loss has no direct p dependence and x_0 is fixed, so its two non-sum terms are zero. Adding the three entries in the last column gives

$$\frac{dJ}{dp} = -0.0332658376704.$$

Those zero terms are a feature of this chosen calculation, not a general rule: a parameter-dependent observation/loss or parameter-dependent initial state contributes through the first or last term of the total-derivative formula.

The accompanying script independently evaluates the same recurrence with PyTorch and a central finite difference. Its retained receipt records

$$\begin{aligned} \text{reverse accumulation} &= -0.0332658376704, \\ \text{PyTorch autograd} &= -0.0332658376704, \\ \text{central difference } (h = 10^{-6}) &= -0.0332658376618. \end{aligned}$$

The small finite-difference discrepancy is truncation and floating-point roundoff, not a new model effect. The complete check takes under five seconds on the recorded CPU and is reproducible with:

```
python source/book/scripts/build_backprop_example.py
```

This command is written for the extracted public edition, from its package root; the script resolves `source/` as its course root and writes only its figure and review receipt there.

The machine-readable validation receipt retains the measured runtime, all forward/reverse values, and the analytic, automatic-differentiation, and central-difference assertions used here.

The check confirms this scalar recurrence and its stated arithmetic. It does not validate a fracture derivative or a complete optimisation workflow.

A restricted fixed-history AT2 connection

The packaged public PhAST source documents an AT2 damage weak form and an adjoint CG backward for selected damage-solve inputs in `vendor/PhAST/src/phast/solvers/damage_solver.py`. For an **unconstrained interior solve** (or a residual restricted to free damage degrees of freedom), with spatially constant G_c and no mesh-dependent γ correction, a useful **fixed-history, fixed-mechanics reduced** notation for that damage subproblem is

$$R_d(d; G_c, H) = \left[\frac{G_c}{\ell} M + G_c \ell K + 2M_H\right] d - 2b_H = 0,$$

where M and K are mass and stiffness assemblies, while M_H and b_H collect the history-weighted assembly. With H, mechanics, and ℓ held fixed, let

$$A = \frac{\partial R_d}{\partial d}, \qquad A^{\mathsf{T}}\lambda = \frac{\partial J}{\partial d}.$$

Implicit differentiation of this *reduced* residual gives

$$\frac{dJ}{dG_c} = \frac{\partial J}{\partial G_c} - \lambda^{\mathsf{T}}\left[\left(\frac{M}{\ell} + \ell K\right) d\right].$$

The direct term remains only when the chosen loss explicitly depends on G_c. The minus sign follows from differentiating $R_d = 0$ and solving for dd/dG_c.

This is intentionally narrower than an end-to-end fracture claim. In a full load history, mechanics can change H; an irreversibility update may use a non-smooth maximum; damage bounds can activate; and nonlinear/staggered iterations and stopping criteria matter. Active-set changes, history changes, and convergence failures must be accounted for by the selected differentiation rule. A discrete model change is not automatically differentiable merely because it is written in tensor operations.

Check your understanding

1. In the scalar example, why are there three $B_n\lambda_{n+1}$ terms even though there is only one scalar parameter p?
2. Suppose $x_0 = x_0(p)$ instead of being fixed. Which term in the total derivative changes, and why is it propagated by λ_0 rather than λ_3?
3. Which assumptions must remain fixed before the displayed AT2 G_c identity can be treated as a reduced-subproblem calculation?

Short answers

1. The single parameter is used once in each of the three update maps. Each local use affects the final loss through the remaining steps, so reverse mode adds one local parameter contribution for each use.
2. The initial-state term $(\partial z_0/\partial p)^{\mathsf{T}}\lambda_0$ becomes nonzero. The adjoint at step zero contains the sensitivity of the final loss to a perturbation of the initial state after every later update has been traversed backwards.
3. The mechanics-dependent history field H, the mechanics state, and ℓ are held fixed. The identity is not by itself a statement about history maxima, bounds/active sets, staggered convergence, or a total derivative through a full fracture load path.

6.4 A bounded parameterisation

An unconstrained update can turn a positive material multiplier negative. One simple way to enforce a scalar interval is to optimise an unconstrained variable θ and map it to

$$p(\theta) = p_{\min} + (p_{\max} - p_{\min})\, s(\theta), \qquad s(\theta) = \frac{1}{1 + \exp(-\theta)}.$$

The chain rule gives

$$\frac{\mathrm{d}J}{\mathrm{d}\theta} = \frac{\mathrm{d}J}{\mathrm{d}p}(p_{\max} - p_{\min})s(\theta)(1 - s(\theta)).$$

This construction enforces bounds but introduces saturation near the limits: the sigmoid derivative becomes small. A bounded transform is a modelling and optimisation decision, not a guarantee that the inverse problem is well conditioned.

6.5 A minimal inverse experiment

For a first experiment, choose one parameter, one geometry, one loading protocol, and one stated observable. For example:

1. define a reference parameter p_{ref} within a known interval;
2. compute a synthetic observation $y^\star = y(p_{\text{ref}})$ using a documented forward map;
3. start from a different θ and map it to $p(\theta)$;
4. compute $J(\theta)$ and a gradient;
5. take a bounded update; and
6. report the parameter trajectory, loss, observable mismatch, and a derivative check.

This teaches recovery in a controlled setting. It does not validate recovery from experimental data, because synthetic data share the forward model and noise assumptions used to generate them.

6.6 Computational lesson: check a derivative and recover a positive modulus

The lesson below is an original one-dimensional elastic-bar toy. It first compares automatic differentiation, a central finite difference, and the known analytic sensitivity; it then recovers a synthetic modulus and checks a held-out load case. Its implementation uses a positive log-modulus map, $E = \exp(\theta)$, rather than the finite-interval sigmoid map in the preceding section. Positivity is therefore enforced in the lesson, while finite lower and upper bounds remain a separate parameterisation choice to assess for a particular inverse problem.

It is not a PhAST fracture inverse and should not be used to infer differentiability through damage history, irreversibility, active sets, or a nonlinear fracture solve.

6.6.1 Differentiate and recover a modulus in the bar toy

Learning objective. Derive the exact tip-displacement sensitivity of the course-owned elastic-bar exercise, then audit its positive modulus parameterisation and held-out recovery evidence.

Predict first. For a fixed positive load, predict the sign of $\mathrm{d}u_{\text{tip}}/\mathrm{d}E$. Also predict whether the notebook's exp(log_E) parameterisation imposes an upper bound on the modulus.

Recorded computation: 5.446 seconds on the reference local CPU after setup. This is not a fresh Colab timing.

Read the code and its recorded outputs in order. To change inputs and run Python, download a notebook and use the course environment. The static book does not execute these cells in the browser.

Download practice notebook · Download with worked solutions · Environment setup

This notebook is an original one-dimensional elastic-bar tensor exercise. It is **not a PhAST fracture inverse**, does not use research data, and should not be used to infer the differentiability of a history-dependent damage calculation. It makes the chain explicit: trainable modulus → stiffness tensor → linear solve → observed displacement → scalar loss.

```
from pathlib import Path
import sys

def locate_course_root():
    for base in (Path.cwd(), *Path.cwd().parents, Path(__file__).resolve().parent␣
↪if "__file__" in globals() else Path.cwd()):
        direct = base / "teaching" / "ukacm_autumn_school_2026"
        if (direct / "notebooks" / "day2_helpers").is_dir():
            return direct
        if (base / "notebooks" / "day2_helpers").is_dir():
            return base
    raise FileNotFoundError("Could not find teaching/ukacm_autumn_school_2026.")

COURSE_ROOT = locate_course_root()
if str(COURSE_ROOT / "notebooks") not in sys.path:
    sys.path.insert(0, str(COURSE_ROOT / "notebooks"))

import matplotlib.pyplot as plt
import numpy as np
import torch
from io import BytesIO
from IPython.display import Image, display

from day2_helpers import assets_dir

torch.set_default_dtype(torch.float64)

def display_figure(figure, dpi=150, alt="Rendered teaching figure"):
    """Embed a PNG even when nbclient uses a non-interactive Agg backend."""
    buffer = BytesIO()
    figure.savefig(buffer, format="png", dpi=dpi, bbox_inches="tight")
    display(Image(data=buffer.getvalue(), alt=alt))

print({"course_layout": "teaching/ukacm_autumn_school_2026 located", "torch":␣
↪torch.__version__, "device": "cpu", "dtype": str(torch.get_default_dtype())})
```

```
{'course_layout': 'teaching/ukacm_autumn_school_2026 located', 'torch': '2.8.0',
↪'device': 'cpu', 'dtype': 'torch.float64'}
```

```
torch.manual_seed(20260909)
n_elements, length, area = 40, 1.0, 1.0
h = length / n_elements
# Unit-modulus stiffness. E remains a differentiable scalar multiplier.
K0 = torch.zeros((n_elements + 1, n_elements + 1), dtype=torch.float64)
local = torch.tensor([[1.0, -1.0], [-1.0, 1.0]], dtype=torch.float64) * area / h
for e in range(n_elements):
    K0[e:e+2, e:e+2] += local

def tip_displacement(E, load):
    K_free = (E * K0)[1:, 1:]
    force = torch.zeros(n_elements, dtype=torch.float64)
    force[-1] = load
    return torch.linalg.solve(K_free, force)[-1]

E_probe = torch.tensor(1.8, dtype=torch.float64, requires_grad=True)
u_probe = tip_displacement(E_probe, 1.1)
(du_dE_ad,) = torch.autograd.grad(u_probe, E_probe)
```

(continues on next page)

(continued from previous page)

```
h_fd = 1.0e-6
du_dE_fd = (tip_displacement(E_probe.detach() + h_fd, 1.1) - tip_displacement(E_
↪probe.detach() - h_fd, 1.1)) / (2*h_fd)
du_dE_exact = -u_probe.detach() / E_probe.detach()
print({"u_tip": float(u_probe.detach()), "AD": float(du_dE_ad), "FD": float(du_dE_
↪fd), "analytic": float(du_dE_exact)})
assert abs(float(du_dE_ad - du_dE_exact)) < 1.0e-12
assert abs(float(du_dE_fd - du_dE_exact)) < 1.0e-8
```

```
{'u_tip': 0.6111111111111187, 'AD': -0.33950617283949214, 'FD': -0.
↪3395061727862192, 'analytic': -0.3395061728395104}
```

```
# Whole load cases are kept separate: train, validation, and held-out.
E_true = torch.tensor(2.4, dtype=torch.float64)
train_loads = torch.tensor([0.35, 0.75, 1.15, 1.55], dtype=torch.float64)
validation_load = torch.tensor(0.95, dtype=torch.float64)
heldout_load = torch.tensor(1.35, dtype=torch.float64)
observed_train = torch.stack([tip_displacement(E_true, load) for load in train_
↪loads]).detach()
observed_validation = tip_displacement(E_true, validation_load).detach()
observed_heldout = tip_displacement(E_true, heldout_load).detach()

log_E = torch.nn.Parameter(torch.log(torch.tensor(0.9, dtype=torch.float64)))
optimiser = torch.optim.Adam([log_E], lr=0.08)
history = []
for iteration in range(180):
    optimiser.zero_grad(set_to_none=True)
    E_trial = torch.exp(log_E)
    predicted = torch.stack([tip_displacement(E_trial, load) for load in train_
↪loads])
    loss = torch.mean((predicted - observed_train) ** 2)
    loss.backward()
    optimiser.step()
    if iteration % 10 == 0 or iteration == 179:
        with torch.no_grad():
            val_error = abs(tip_displacement(torch.exp(log_E), validation_load) -␣
↪observed_validation)
        history.append((iteration + 1, float(torch.exp(log_E).detach()),␣
↪float(loss.detach()), float(val_error)))

E_fit = torch.exp(log_E).detach()
heldout_prediction = tip_displacement(E_fit, heldout_load)
print({"E_true": float(E_true), "E_fit": float(E_fit), "heldout_observed":␣
↪float(observed_heldout), "heldout_prediction": float(heldout_prediction)})
assert abs(float(E_fit - E_true)) < 2.0e-3
```

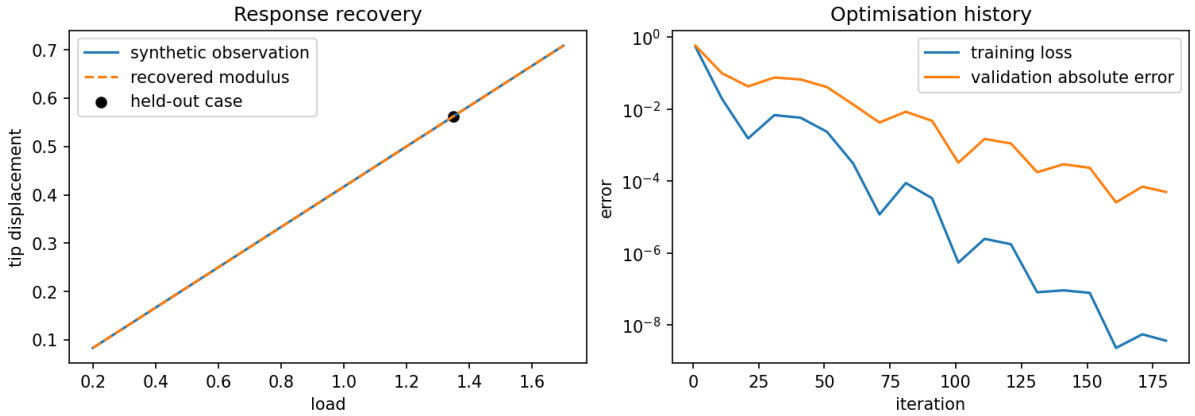
```
{'E_true': 2.4, 'E_fit': 2.400302387627723, 'heldout_observed': 0.562499999999992,
↪'heldout_prediction': 0.5624291368281488}
```

```
history_array = np.array(history)
response_loads = torch.linspace(0.2, 1.7, 80, dtype=torch.float64)
true_response = np.array([float(tip_displacement(E_true, load)) for load in␣
↪response_loads])
fit_response = np.array([float(tip_displacement(E_fit, load)) for load in response_
↪loads])
fig, axes = plt.subplots(1, 2, figsize=(10, 3.5), constrained_layout=True)
axes[0].plot(response_loads, true_response, label="synthetic observation")
axes[0].plot(response_loads, fit_response, "--", label="recovered modulus")
axes[0].scatter([heldout_load], [observed_heldout], color="black", label="held-out␣
↪case")
```

(continues on next page)

(continued from previous page)

```
axes[0].set(xlabel="load", ylabel="tip displacement", title="Response recovery")
axes[0].legend()
axes[1].semilogy(history_array[:, 0], history_array[:, 2], label="training loss")
axes[1].semilogy(history_array[:, 0], history_array[:, 3], label="validation␣
↪absolute error")
axes[1].set(xlabel="iteration", ylabel="error", title="Optimisation history")
axes[1].legend()
fig.savefig(assets_dir() / "tiny_inverse_bar.png", dpi=160, bbox_inches="tight")
display_figure(fig, alt="Toy elastic-bar response recovery and optimisation␣
↪history.")
plt.close(fig)
```



The bar is intentionally simple enough that its exact sensitivity is known. In a fracture inverse exercise, nonsmooth history maxima, irreversibility constraints, load path, solver tolerance, and the observation design require additional verification. This toy gives learners a short place to diagnose AD versus finite differences before encountering those complications.

Exercises

Try each question before opening the hint or worked solution. Keep the original calculation as your reference.

Exercise 1: Derive the tip sensitivity

The fixed-left, unit-length, unit-area bar has a tip load F and a uniform modulus E.

1. Derive $u_{\text{tip}}(E, F)$ and $\mathrm{d}u_{\text{tip}}/\mathrm{d}E$.
2. Evaluate both at $E = 1.8$ and $F = 1.1$.
3. Compare with the notebook's autograd and centred-FD values.

Hint 1

For a uniform axial bar, the series assembly gives $u_{\text{tip}} = FL/(EA)$. Here $L = A = 1$.

Worked solution 1

1. With $L = A = 1$, the assembled bar has

$$u_{\text{tip}} = \frac{F}{E}.$$

Differentiating gives

$$\frac{\mathrm{d}u_{\text{tip}}}{\mathrm{d}E} = -\frac{F}{E^2} = -\frac{u_{\text{tip}}}{E}.$$

2. Therefore
$$u_{\text{tip}} = \frac{1.1}{1.8} = 0.6111111111, \qquad \frac{\mathrm{d}u_{\text{tip}}}{\mathrm{d}E} = -\frac{1.1}{1.8^2} = -0.3395061728.$$
3. The retained autograd value is -0.33950617283949214, the centred-FD value is -0.3395061727862192, and the analytic value is -0.3395061728395104. The negative sign has the expected mechanics interpretation: a stiffer bar displaces less under the same load. This calculation is the course-owned bar toy, not a phase-field fracture sensitivity.

Exercise 2: Audit the positive recovery parameterisation

The optimiser holds **log_E** as its trainable parameter and computes $E_{\text{trial}} = \exp(\text{log\_E})$, starting from $E = 0.9$. It reports $E_{\text{true}} = 2.4$, $E_{\text{fit}} = 2.4003023876$, held-out observed displacement 0.5625, and held-out prediction 0.5624291368.

1. What values of E can this parameterisation represent?
2. Why is it incorrect to describe this code as a bounded-sigmoid parameterisation?
3. Compute the absolute held-out displacement error and state one reason the held-out load is useful.

Hint 2

The exponential maps every real number to a strictly positive number. Compare the observed and predicted held-out displacements directly.

Worked solution 2

1. Since $\exp(z) > 0$ for every real z, the parameterisation enforces only $E > 0$. It has no finite upper bound. The initial value is obtained because $\exp(\log(0.9)) = 0.9$.
2. A sigmoid would constrain a raw parameter to $(0, 1)$ before any explicit scaling. The canonical notebook uses $\exp(\text{log\_E})$, not a sigmoid, so it should be described as a positivity transform only.
3. The held-out absolute error is
$$|0.562499999999992 - 0.5624291368281488| = 7.0863 \times 10^{-5}.$$

The held-out load 1.35 was not one of the four optimisation loads, so it checks whether the recovered scalar predicts a separate observation rather than merely reproducing the fitting points. It remains a deliberately simple inverse toy: a successful scalar recovery does not validate a history-dependent fracture inverse.

6.7 A directional derivative check

Let g be a gradient returned by automatic differentiation. Compare
$$D_h J(p; q) \quad \text{with} \quad g^{\mathsf{T}} q$$
over a short sweep of reasonable h values. A useful result card shows the direction q, the h values, the two numbers, and an error measure such as
$$\frac{|D_h J - g^{\mathsf{T}} q|}{\max(1, |D_h J|, |g^{\mathsf{T}} q|)}.$$
Do not choose a single step only because it happens to agree. If the check fails, investigate tensor broadcasting, parameter detachment, in-place updates, inconsistent normalisation, solver tolerances, and non-smooth branches before interpreting an optimisation trajectory.

What a passed check means

A small discrepancy at one (p, q, h) supports the local derivative of the implemented scalar computation. It does not prove global smoothness, convergence of the nonlinear forward problem, correct physical calibration, or uniqueness of an inverse solution.

6.8 Why inverse recovery can be difficult

Several different parameter fields can produce similar limited observations. This non-uniqueness can arise from insufficient loading diversity, a coarse observable, parameter correlation, regularisation choices, or weak sensitivity of the forward response. A decreasing loss should therefore be reported with the parameter path, held-out or alternative observables where available, and the stated prior/bounds.

For a spatial parameter field $p(x)$, an additional smoothness term might be

$$J_{\text{reg}}(p) = J_{\text{data}}(p) + \frac{\beta}{2} \int_{\Omega} |\nabla p|^2 \, \mathrm{d}x.$$

This selects smoother solutions, but it also changes the inverse problem. The coefficient β is not a harmless plotting setting.

6.9 Exercise: interpret a gradient result

An implementation reports a small central-difference discrepancy for one direction q at a parameter p. Which claims below are justified?

1. The local scalar derivative is consistent with this finite-difference check.
2. The inverse problem has a unique solution.
3. The phase-field model is calibrated to a real specimen.
4. The automatic-differentiation graph likely connects the chosen parameter to the stated loss along this tested path.

Solution

Claims 1 and 4 are justified, with the qualification that the check is local and depends on the selected step, direction, and forward solve. Claims 2 and 3 need additional evidence: identifiability requires a study of observations, parameters, and priors; calibration requires suitable experimental comparison and uncertainty treatment.

The computational lesson makes the same distinction in a one-dimensional elastic bar. Its passed comparison concerns a positive log-modulus parameterisation and the displayed scalar observable, not a finite-bound transform or a phase-field fracture inverse.

6.10 Sources for this chapter

The PyTorch autograd documentation and JAX automatic-differentiation guide describe their respective differentiation systems. For a finite-element example of implicit gradient computation, see the JAX-FEM gradient example.

CHAPTER

SEVEN

TRAIN, SAVE, RELOAD, ADAPTERS, AND CHECKED PROPOSALS

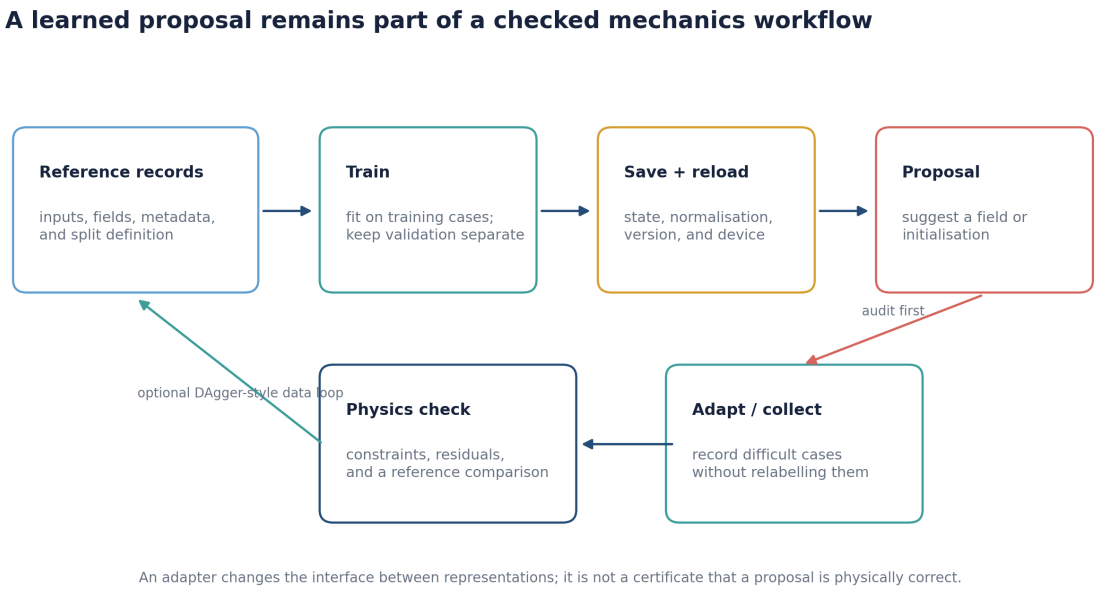


Figure 7.1: A learning component is useful only when its inputs, outputs, and physics checks are stated as clearly as those of the reference calculation.

7.1 Decide what is being learned

The phrase “use machine learning for fracture” hides several different tasks. Name the role precisely.

- **Surrogate:** approximate a map from stated inputs to a stated output.
- **Proposal model:** supply an initial damage field, displacement field, or parameter guess to a mechanics calculation.
- **Adapter:** convert one data representation to another, such as a mesh ordering or normalised tensor layout.
- **Corrector:** update a proposal using a residual, an iterative solve, or a trusted reference computation.
- **Emulator for an observable:** approximate a scalar response while not necessarily reproducing a full field.

These roles can be combined, but a label should not claim more than the interface actually supports. A model that predicts a useful initialisation is not automatically a replacement for a fracture solver.

7.2 A small supervised-learning contract

Let x_i be an input representation and z_i a reference target. A simple training objective is

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \|f_\theta(x_i) - z_i\|_2^2 .$$

Before training, write down:

1. what each channel of x_i means, including coordinates, parameters, fields, and units;
2. what target z_i means and at which load step or time it is sampled;
3. how meshes, nodes, or quadrature points are ordered;
4. the training, validation, and test split rule; and
5. the metric and physical diagnostic reported on each split.

Normalisation belongs to this contract. If an input component is transformed as $\tilde{x} = (x - \mu)/s$, save μ and s with the model. A reloaded model without its preprocessing state is a different computational map.

7.3 Where does the training gradient travel?

Distinguish three computational graphs before choosing a learning method.

Supervised fitting compares a prediction with stored targets:

$$\mathcal{L}_{\text{sup}} = \|f_\theta(x) - z_{\text{ref}}\|^2.$$

A solver may have generated the targets offline. Its derivatives are not needed for this training graph.

Residual-based training evaluates a stated equation at the prediction:

$$\mathcal{L}_{\text{res}} = \|R(f_\theta(x); p)\|^2.$$

The gradient passes through the residual evaluation. This does not necessarily differentiate a converged solve. Boundary conditions and branch selection still matter, as the opening algebraic teaser illustrates.

Solver-in-the-loop training evaluates the consequences of a prediction:

$$z_0 = f_\theta(x), \qquad z_{n+1} = S(z_n, p), \qquad \mathcal{L} = \|C z_N - y_*\|^2.$$

Here C extracts the observed quantity. With fixed p and a loss depending only on z_N, reverse propagation gives

$$\bar{z}_N = 2C^T(Cz_N - y_*), \qquad \bar{z}_n = \left(\frac{\partial S}{\partial z_n}\right)^T \bar{z}_{n+1}, \qquad \nabla_\theta \mathcal{L} = \left(\frac{\partial f_\theta}{\partial \theta}\right)^T \bar{z}_0.$$

These are vector–Jacobian products; the full Jacobians need not be stored. An implicit adjoint may replace differentiation through every solver iteration. The differentiable-physics introduction develops this operator viewpoint. For fracture, history updates, active constraints, stopping tolerances and any detached arrays require explicit derivative conventions and checks.

The following notebooks demonstrate supervised training and checked proposals on a toy field problem. They do not yet implement end-to-end training through the PhAST fracture solver.

7.4 Train, save, and reload reproducibly

A minimal saved model package should include:

- model weights and architecture configuration;
- input and target normalisation parameters;
- feature/channel names and tensor-shape convention;

- training data identifier and split definition;
- software/library version and numerical precision; and
- a small reload test that compares a saved prediction against the in-memory prediction for the same declared input.

7.5 Computational lesson: train, save, reload, and compare a toy field model

The lesson below makes the saved state visible: feature order, normalisation, mesh signature, data split, checkpoint metadata, and a reload comparison all appear beside the code and field plots. Begin by predicting which information must survive serialization for a newly created model object to reproduce the same output. The lesson uses an original `ToyHelmholtzProblem`, not PhAST fracture data or a trained fracture-damage model. It is an interface test, not evidence of a general fracture accelerator.

7.5.1 Train, save, reload, and compare a field model

Learning objective. Use the course-owned ToyHelmholtz field example to reason about whole-case data splits, checkpoint metadata, exact reload agreement, held-out fields, and architecture compatibility.

Predict first. Predict why splitting individual nodes rather than whole load cases can leak information, and whether a successful reload proves that an MLP checkpoint is automatically compatible with a CNN, GNN, or GNO.

Recorded computation: 9.473 seconds on the reference local CPU after setup. This is not a fresh Colab timing.

Read the code and its recorded outputs in order. To change inputs and run Python, download a notebook and use the course environment. The static book does not execute these cells in the browser.

Download practice notebook · Download with worked solutions · Environment setup

We train one small MLP on an original, course-owned `ToyHelmholtzProblem`. It is a fast field equation with a known discrete residual, designed to demonstrate data splits, checkpoint metadata, and a held-out field comparison. It is **not PhAST fracture data**, a trained public `learned_damage` model, or a claim of fracture acceleration.

Feature contract: nodewise `[x/L, y/H, load_factor]`, all dimensionless, in exactly that order; output: one nodewise damage-like proposal in `[0,1]`. Whole load cases—not individual nodes—are separated into train/validation/test splits.

```
from pathlib import Path
import sys

def locate_course_root():
    for base in (Path.cwd(), *Path.cwd().parents, Path(__file__).resolve().parent␣
↪if "__file__" in globals() else Path.cwd()):
        direct = base / "teaching" / "ukacm_autumn_school_2026"
        if (direct / "notebooks" / "day2_helpers").is_dir():
            return direct
        if (base / "notebooks" / "day2_helpers").is_dir():
            return base
    raise FileNotFoundError("Could not find teaching/ukacm_autumn_school_2026.")

COURSE_ROOT = locate_course_root()
if str(COURSE_ROOT / "notebooks") not in sys.path:
    sys.path.insert(0, str(COURSE_ROOT / "notebooks"))

import matplotlib.pyplot as plt
import numpy as np
import torch
from io import BytesIO
from IPython.display import Image, display

from day2_helpers import assets_dir
```

(continues on next page)

(continued from previous page)

```
torch.set_default_dtype(torch.float64)

def display_figure(figure, dpi=150, alt="Rendered teaching figure"):
    """Embed a PNG even when nbclient uses a non-interactive Agg backend."""
    buffer = BytesIO()
    figure.savefig(buffer, format="png", dpi=dpi, bbox_inches="tight")
    display(Image(data=buffer.getvalue(), alt=alt))

print({"course_layout": "teaching/ukacm_autumn_school_2026 located", "torch":␣
↪torch.__version__, "device": "cpu", "dtype": str(torch.get_default_dtype())})
```

```
{'course_layout': 'teaching/ukacm_autumn_school_2026 located', 'torch': '2.8.0',
↪'device': 'cpu', 'dtype': 'torch.float64'}
```

```
from day2_helpers.course_tools import (
    FEATURE_ORDER, TinyDamageMLP, ToyHelmholtzProblem,
    load_toy_model, make_toy_checkpoint, normalise_features,
)

checkpoint_path = assets_dir() / "models" / "tiny_damage_mlp.pt"
checkpoint = make_toy_checkpoint(checkpoint_path, epochs=400, force=True)
metadata = checkpoint["metadata"]
print({"checkpoint": str(checkpoint_path.relative_to(COURSE_ROOT)), "metadata":␣
↪metadata, "history": checkpoint["history"], "metrics": checkpoint["metrics"]})
```

```
{'checkpoint': 'assets/day2_forward/models/tiny_damage_mlp.pt', 'metadata': {
↪'interface_version': 'ukacm-toy-damage-v1', 'architecture': 'TinyDamageMLP',
↪'width': 32, 'feature_order': ['x_over_L', 'y_over_H', 'load_factor'], 'feature_
↪location': 'node', 'feature_units': ['dimensionless x/L', 'dimensionless y/H',
↪'dimensionless'], 'output': 'toy_damage_proposal at nodes; d=0 intact-like, d=1␣
↪saturated', 'dtype': 'float64', 'device': 'cpu', 'mesh_signature': [25, 13],
↪'normalisation_mean': [0.5, 0.5, 0.4000000000000001], 'normalisation_scale': [0.
↪3005011345450644, 0.3118447648923806, 0.20087244715634883], 'splits': {'train':␣
↪[0.12, 0.17818181818181816, 0.23636363636363636, 0.29454545454545455, 0.
↪3527272727272727, 0.4109090909090909, 0.4690909090909091, 0.5272727272727273, 0.
↪5854545454545454, 0.6436363636363637, 0.7018181818181818, 0.76], 'validation':␣
↪[0.82, 0.88], 'test': [0.94, 1.0]}, 'data_provenance': 'course-owned␣
↪ToyHelmholtzProblem; not PhAST or research data', 'seed': 20260909, 'epochs':␣
↪400}, 'history': [{'epoch': 1.0, 'train_mse': 0.1517765063048665, 'validation_mse
↪': 0.07862736928885466}, {'epoch': 25.0, 'train_mse': 0.009012028414098669,
↪'validation_mse': 0.025264187641007}, {'epoch': 100.0, 'train_mse': 0.
↪0012914382086734117, 'validation_mse': 0.005490918863171005}, {'epoch': 400.0,
↪'train_mse': 0.0001303170500437651, 'validation_mse': 0.00078929913555557108}],
↪'metrics': {'test_rmse': 0.049867531488768393, 'training_seconds': 1.
↪3265216250001686}}
```

```
# Reload into a fresh model instance; compare the two predictions exactly.
problem = ToyHelmholtzProblem()
heldout_load = 1.00
heldout_features = problem.features(heldout_load)
heldout_reference = problem.reference_field(heldout_load)
mean = torch.tensor(metadata["normalisation_mean"], dtype=torch.float64)
scale = torch.tensor(metadata["normalisation_scale"], dtype=torch.float64)
original = TinyDamageMLP(width=metadata["width"]).to(dtype=torch.float64)
original.load_state_dict(checkpoint["state_dict"])
original.eval()
reloaded, reloaded_metadata = load_toy_model(checkpoint_path)
with torch.no_grad():
    prediction_before = original(normalise_features(heldout_features, mean,␣
```

(continues on next page)

(continued from previous page)

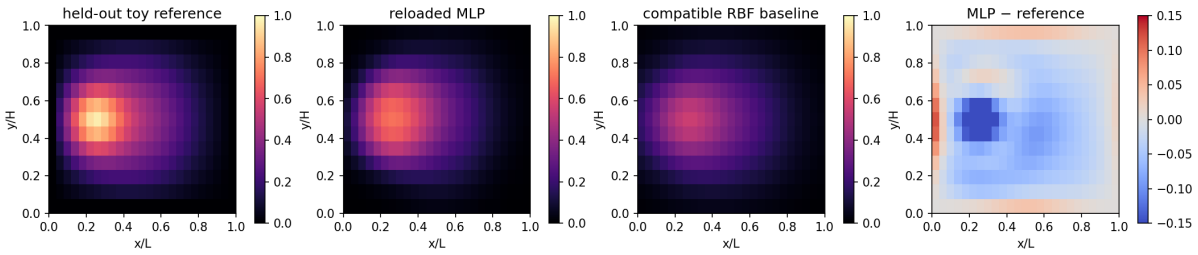
```
↪scale)).squeeze(1)
    prediction_after = reloaded(normalise_features(heldout_features, mean, scale)).
↪squeeze(1)
reload_difference = float((prediction_before - prediction_after).abs().max())
heldout_rmse = float(torch.sqrt(torch.mean((prediction_after - heldout_reference)␣
↪** 2)))
print({"reload_max_abs_difference": reload_difference, "heldout_rmse": heldout_
↪rmse, "feature_order": reloaded_metadata["feature_order"]})
assert reload_difference < 1.0e-12
assert heldout_rmse < 0.07
```

```
{'reload_max_abs_difference': 0.0, 'heldout_rmse': 0.05636315718170472, 'feature_
↪order': ['x_over_L', 'y_over_H', 'load_factor']}
```

```
# A compatible RBF reference alternative shares the same nodewise features,
# but is a distinct architecture with its own resolution/data assumptions.
train_loads = metadata["splits"]["train"]
train_features, train_targets = problem.case_tensor(train_loads)
z_train = normalise_features(train_features, mean, scale)
z_test = normalise_features(heldout_features, mean, scale)
squared_distance = torch.cdist(z_test, z_train) ** 2
weights = torch.exp(-squared_distance / 0.18)
rbf_prediction = (weights @ train_targets).squeeze(1) / weights.sum(dim=1).clamp_
↪min(1.0e-15)
rbf_rmse = float(torch.sqrt(torch.mean((rbf_prediction - heldout_reference) ** 2)))
print({"MLP_heldout_RMSE": heldout_rmse, "RBF_heldout_RMSE": rbf_rmse, "RBF_scope
↪": "same toy feature contract only"})
```

```
{'MLP_heldout_RMSE': 0.05636315718170472, 'RBF_heldout_RMSE': 0.0997143167782659,
↪'RBF_scope': 'same toy feature contract only'}
```

```
def field_image(values):
    return values.detach().reshape(problem.ny, problem.nx).cpu().numpy()

error = prediction_after - heldout_reference
fig, axes = plt.subplots(1, 4, figsize=(14, 3.2), constrained_layout=True)
for ax, values, title, cmap, limits in (
    (axes[0], heldout_reference, "held-out toy reference", "magma", (0, 1)),
    (axes[1], prediction_after, "reloaded MLP", "magma", (0, 1)),
    (axes[2], rbf_prediction, "compatible RBF baseline", "magma", (0, 1)),
    (axes[3], error, "MLP - reference", "coolwarm", (-0.15, 0.15)),
):
    image = ax.imshow(field_image(values), origin="lower", extent=(0,1,0,1),␣
↪cmap=cmap, vmin=limits[0], vmax=limits[1])
    ax.set(title=title, xlabel="x/L", ylabel="y/H", aspect="equal")
    fig.colorbar(image, ax=ax, shrink=0.78)
fig.savefig(assets_dir() / "tiny_mlp_heldout_field.png", dpi=160, bbox_inches=
↪"tight")
display_figure(fig, alt="Held-out toy reference, reloaded MLP field, compatible␣
↪RBF field, and MLP error.")
plt.close(fig)
```



The checkpoint stores the architecture width, weight `state_dict`, feature order and units, normalisation, dtype/device, mesh signature, data provenance, splits, seed, and training duration. An MLP and RBF can both consume this *toy* nodewise representation; a CNN would require mesh-to-grid maps, and a GNN/GNO would require connectivity/operator inputs. A model name alone is not a compatibility guarantee.

Exercises

Try each question before opening the hint or worked solution. Keep the original calculation as your reference.

Exercise 1: Count examples without leaking a load case

The toy grid has mesh signature 25×13. The checkpoint uses 12 whole load cases for training, 2 for validation, and 2 for testing. Each case supplies one nodewise feature vector $[x/L, y/H, \text{load factor}]$ at every node.

1. Count the nodes per load case and the nodewise rows in each split.
2. Explain why the split is made by whole load case rather than by individual node.
3. Is the held-out field at load factor 1.00 a training example in this receipt?

Hint 1

Multiply the grid dimensions first. Then multiply the result by the number of cases in each split. Check the metadata: the test loads are 0.94 and 1.00.

Worked solution 1

1. One field has

$$25 \times 13 = 325$$

nodes. Therefore the training split has $12 \times 325 = 3900$ nodewise rows, while validation and test each have $2 \times 325 = 650$ rows.

2. Nodes from one load case share the same field and load context. If some nodes from a field appeared in training while other nodes from that same field appeared in test, the measured test error would partly reflect interpolation within a seen case rather than generalisation to an unseen load case. The canonical exercise therefore keeps entire loads together.
3. No. The checkpoint metadata lists test loads 0.94 and 1.00, so the notebook's held-out comparison at 1.00 uses a test case. This still concerns a tiny course-owned field problem, not fracture data.

Exercise 2: Interpret reload agreement and the MLP/RBF comparison

The retained output gives reload maximum absolute difference 0, held-out MLP RMSE 0.0563631572, and held-out RBF RMSE 0.0997143168.

1. Compute the RMSE improvement of the MLP over the RBF on this held-out field.
2. List four checkpoint/interface facts that must be checked before inference.
3. State what reload difference 0 verifies, and one thing it does not verify.

Hint 2

Subtract the two RMSE values for the absolute gap. Read the metadata card: it records architecture, feature order, normalisation, and mesh information.

Worked solution 2

1. The absolute held-out RMSE gap is

$$0.0997143168 - 0.0563631572 = 0.0433511596.$$

For this one toy held-out field, the MLP has the lower RMSE. It is not a comparison of fracture accuracy or wall-clock acceleration.

2. Examples of required facts are: the architecture identifier and width; the **state_dict** weights; exact feature order $[x/L, y/H,$ load factor$]$ and units; normalisation mean and scale; nodewise feature location; dtype/device; mesh signature $[25, 13]$; the training-data provenance and supported scope. Four is the minimum here, not an exhaustive list.
3. A reload difference of 0 verifies that a fresh instance reconstructed from the saved metadata and weights produced the same prediction for this tested input. It does not establish robustness on other meshes, physical correctness, a compatible CNN/GNN/GNO representation, or a speedup. A CNN needs explicit mesh-to-grid and grid-to-mesh maps; a GNN/GNO needs graph or operator information.

Reload test

Choose one fixed input record. Run the model before saving, reload into a new model object, run the same record, and compare the output after applying the same preprocessing. If the values differ, inspect normalisation, device/dtype, model mode, feature order, and serialised configuration before training again.

7.6 What an adapter can and cannot repair

An adapter is a stated map between representations,

$$a : \mathcal{X}_{\text{source}} \longrightarrow \mathcal{X}_{\text{target}}.$$

For example, it may reorder nodes, concatenate a coordinate channel, project a cell-centred value to nodes, or convert units. These operations can be necessary and useful. They cannot manufacture missing information or correct a target whose physical meaning differs from the source.

Ask four questions before accepting an adapter:

- Does it preserve the geometry and coordinate frame?
- Does it preserve the field meaning, units, and load/time index?
- Does it preserve or deliberately change interpolation order?
- Is there a small round-trip or reference example that checks the mapping?

The word “adapter” should not conceal a change from one mesh, material parameterisation, or loading regime to another. That is a transfer assumption which needs its own evaluation.

7.7 From a proposal to a checked hybrid calculation

Suppose a learned model proposes $\widehat{d}$ for a damage field. A conservative hybrid workflow is

$$\text{input} \longrightarrow \widehat{d} \longrightarrow \text{bounds/residual checks} \longrightarrow \text{mechanics correction or fallback} \longrightarrow \text{reported output}.$$

The checks might include damage bounds, irreversibility relative to the prior state, a mechanics residual, and a comparison to a reference solve on a predefined evaluation set. If a correction is used, report the cost and result of the full route, including prediction, conversion, checks, correction, and any fallback. Bypassing a solver stage is not automatically a wall-time improvement.

7.8 Computational lesson: accept a compatible proposal or fall back

The next lesson reuses the toy field-model contract to test a compatible proposal, reject a deliberately corrupted proposal by a stated residual gate, and fall back to the reference field. Read the compatibility and residual checks before treating the four field panels as evidence. The residual belongs to the course-owned `ToyHelmholtzProblem`; it is not a PhAST AT2 fracture residual. This is a teaching pattern for auditing a proposal, not a claim that a learned field is an admissible fracture solution in general.

7.8.1 Assess a proposal and use a reference correction

Learning objective. Apply the course-owned adapter contract: validate compatible inputs, project bounds and irreversibility, assess the ToyHelmholtz residual, and distinguish one correction record from an end-to-end DAgger result.

Predict first. Predict which of the retained proposals passes a residual limit of 0.30, and whether a field with a lower residual than the limit should still be silently accepted when its feature order is incompatible.

Recorded computation: 5.179 seconds on the reference local CPU after setup. This is not a fresh Colab timing.

Read the code and its recorded outputs in order. To change inputs and run Python, download a notebook and use the course environment. The static book does not execute these cells in the browser.

Download practice notebook · Download with worked solutions · Environment setup

This notebook connects the checkpoint from notebook 04 to a **course-owned teaching adapter**. The adapter verifies its feature order and mesh signature, predicts under `torch.no_grad()`, projects bounds and irreversibility, evaluates the discrete `ToyHelmholtzProblem` residual, and falls back to the reference solve when the proposal fails the gate.

The public PhAST learned-damage hook is an inference interface with detached/no-gradient prediction. This notebook does not train through that hook and its residual is not a PhAST AT2 residual. It teaches the reusable logic: proposal → compatibility → constraint projection → assess → correction/record.

```
from pathlib import Path
import sys

def locate_course_root():
    for base in (Path.cwd(), *Path.cwd().parents, Path(__file__).resolve().parent␣
↪if "__file__" in globals() else Path.cwd()):
        direct = base / "teaching" / "ukacm_autumn_school_2026"
        if (direct / "notebooks" / "day2_helpers").is_dir():
            return direct
        if (base / "notebooks" / "day2_helpers").is_dir():
            return base
    raise FileNotFoundError("Could not find teaching/ukacm_autumn_school_2026.")

COURSE_ROOT = locate_course_root()
if str(COURSE_ROOT / "notebooks") not in sys.path:
    sys.path.insert(0, str(COURSE_ROOT / "notebooks"))

import matplotlib.pyplot as plt
import numpy as np
import torch
from io import BytesIO
from IPython.display import Image, display

from day2_helpers import assets_dir

torch.set_default_dtype(torch.float64)

def display_figure(figure, dpi=150, alt="Rendered teaching figure"):
    """Embed a PNG even when nbclient uses a non-interactive Agg backend."""
```

(continues on next page)

(continued from previous page)

```
    buffer = BytesIO()
    figure.savefig(buffer, format="png", dpi=dpi, bbox_inches="tight")
    display(Image(data=buffer.getvalue(), alt=alt))

print({"course_layout": "teaching/ukacm_autumn_school_2026 located", "torch":␣
↪torch.__version__, "device": "cpu", "dtype": str(torch.get_default_dtype())})
```

```
{'course_layout': 'teaching/ukacm_autumn_school_2026 located', 'torch': '2.8.0',
↪'device': 'cpu', 'dtype': 'torch.float64'}
```

```
import json
from day2_helpers.course_tools import (
    FEATURE_ORDER, ToyDamageAdapter, ToyHelmholtzProblem,
    load_toy_model, make_toy_checkpoint,
)

checkpoint_path = assets_dir() / "models" / "tiny_damage_mlp.pt"
if not checkpoint_path.exists():
    # This makes the notebook independently executable. In the normal
    # sequence notebook 04 has already produced this exact artifact.
    make_toy_checkpoint(checkpoint_path, epochs=400, force=True)
model, metadata = load_toy_model(checkpoint_path)
problem = ToyHelmholtzProblem()
adapter = ToyDamageAdapter(model, metadata, problem)
print({"interface_version": metadata["interface_version"], "feature_order":␣
↪metadata["feature_order"], "mesh_signature": metadata["mesh_signature"]})
```

```
{'interface_version': 'ukacm-toy-damage-v1', 'feature_order': ['x_over_L', 'y_over_
↪H', 'load_factor'], 'mesh_signature': [25, 13]}
```

```
# Compatible held-out request. d_previous comes from a lower-load state.
load_factor = 0.94
d_previous = problem.reference_field(0.82)
proposal = adapter.propose(problem.features(load_factor), d_previous=d_previous)
assessment = adapter.assess(proposal, load_factor, residual_limit=0.30)
reference = problem.reference_field(load_factor)
corrected = proposal if assessment["accepted"] else adapter.fallback(load_factor,␣
↪d_previous)
print({"normal_proposal": assessment, "field_rmse": float(torch.sqrt(torch.
↪mean((proposal-reference)**2)))})
assert assessment["accepted"], assessment

# A deliberately corrupted field is projected but fails the same residual gate.
corrupted = torch.where(
    problem.boundary_mask,
    torch.zeros_like(proposal),
    (proposal + 0.30 * torch.sin(12 * problem.coordinates[:, 0])).clamp(0, 1),
)
rejected = adapter.assess(corrupted, load_factor, residual_limit=0.30)
fallback = adapter.fallback(load_factor, d_previous)
print({"corrupted_proposal": rejected, "fallback_residual": problem.relative_
↪residual(fallback, load_factor)})
assert not rejected["accepted"], rejected
assert problem.relative_residual(fallback, load_factor) < 1.0e-10
```

```
{'normal_proposal': {'toy_relative_residual': 0.19402747874051113, 'residual_limit
↪': 0.3, 'accepted': True, 'scope': 'ToyHelmholtzProblem residual only; not a␣
↪PhAST AT2 residual'}, 'field_rmse': 0.029164814588982523}
{'corrupted_proposal': {'toy_relative_residual': 1.164008876432137, 'residual_limit
```

(continues on next page)

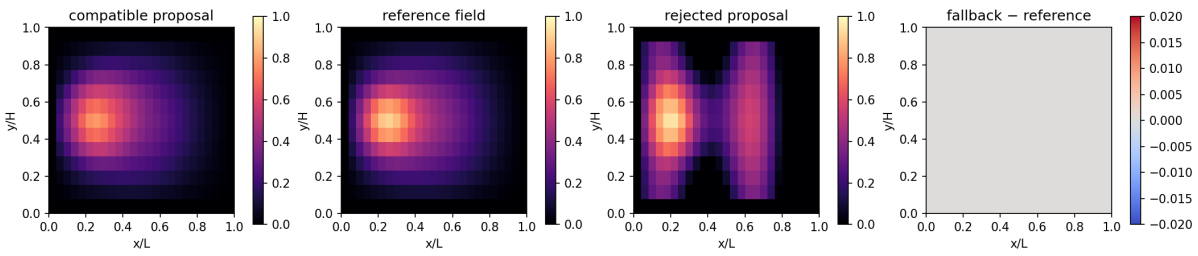
(continued from previous page)

```
↪': 0.3, 'accepted': False, 'scope': 'ToyHelmholtzProblem residual only; not a␣
↪PhAST AT2 residual'}, 'fallback_residual': 2.969565304912253e-15}
```

```
# Explicitly demonstrate an interface failure instead of silently reshaping input.
try:
    adapter.propose(
        problem.features(load_factor),
        feature_order=("load_factor", "x_over_L", "y_over_H"),
    )
except ValueError as error:
    compatibility_message = str(error)
else:
    raise AssertionError("An incompatible feature order was unexpectedly accepted.
↪")
print("Expected incompatibility:", compatibility_message)

nan_features = problem.features(load_factor).clone()
nan_features[0, 0] = float("nan")
try:
    adapter.propose(nan_features)
except ValueError as error:
    finiteness_message = str(error)
else:
    raise AssertionError("A non-finite adapter input was unexpectedly accepted.")
print("Expected finite-input rejection:", finiteness_message)
```

```
Expected incompatibility: Incompatible feature order. Expected ['x_over_L', 'y_
↪over_H', 'load_factor'], got ['load_factor', 'x_over_L', 'y_over_H'].
Expected finite-input rejection: Incompatible feature tensor: all adapter inputs␣
↪must be finite.
```

```
def field_image(values):
    return values.detach().reshape(problem.ny, problem.nx).cpu().numpy()

fig, axes = plt.subplots(1, 4, figsize=(14, 3.2), constrained_layout=True)
panels = (
    (proposal, "compatible proposal", "magma", (0, 1)),
    (reference, "reference field", "magma", (0, 1)),
    (corrupted, "rejected proposal", "magma", (0, 1)),
    (fallback - reference, "fallback - reference", "coolwarm", (-0.02, 0.02)),
)
for ax, field, title, cmap, limits in zip(axes, *zip(*panels)):
    image = ax.imshow(field_image(field), origin="lower", extent=(0,1,0,1),␣
↪cmap=cmap, vmin=limits[0], vmax=limits[1])
    ax.set(title=title, xlabel="x/L", ylabel="y/H", aspect="equal")
    fig.colorbar(image, ax=ax, shrink=0.78)
fig.savefig(assets_dir() / "toy_adapter_fallback.png", dpi=160, bbox_inches="tight
↪")
display_figure(fig, alt="Toy adapter's compatible proposal, reference, rejected␣
↪proposal, and fallback error.")
plt.close(fig)
```



```
# One correction record is a replay-style teaching artefact, not an
# end-to-end DAgger result. Reference-label cost remains part of any
# later retraining comparison.
replay_record = {
    "scope": "course-owned ToyHelmholtzProblem; one offline correction record, not␣
↪PhAST DAgger",
    "load_factor": load_factor,
    "feature_order": list(FEATURE_ORDER),
    "normal_assessment": assessment,
    "rejected_assessment": rejected,
    "correction": "classical ToyHelmholtzProblem reference solve",
    "heldout_evaluation_frozen": True,
}
replay_path = assets_dir() / "toy_adapter_replay_record.json"
replay_path.write_text(json.dumps(replay_record, indent=2), encoding="utf-8")
print({"replay_record": str(replay_path.relative_to(COURSE_ROOT)), "decision":
↪"accept compatible / fallback rejected"})
```

```
{'replay_record': 'assets/day2_forward/toy_adapter_replay_record.json', 'decision
↪': 'accept compatible / fallback rejected'}
```

A genuine offline DAgger cycle would collect model-induced states, obtain reference labels, aggregate only training data, retrain, and reassess a frozen held-out set. The small record above is the transparent first step, not evidence that such a loop has been run for PhAST fracture or that a learned proposal accelerates it.

Exercises

Try each question before opening the hint or worked solution. Keep the original calculation as your reference.

Exercise 1: Make the residual-gate decision

At load factor 0.94, the compatible proposal has relative residual 0.1940274787 and the deliberately corrupted proposal has relative residual 1.1640088764. The acceptance limit is 0.30. The reference fallback has residual 2.9696×10^{-15}.

1. Decide accept or reject for each proposal.
2. Compute the compatible proposal's margin below the limit and the corrupted proposal's excess above it.
3. Explain why the reference result is still needed after a rejection.

Hint 1

The adapter accepts only when residual ≤ 0.30. Subtract in the direction that produces a positive margin or excess.

Worked solution 1

1. The compatible proposal is accepted because

$$0.1940274787 \leq 0.30.$$

The corrupted proposal is rejected because

$$1.1640088764 > 0.30.$$

2. The accepted proposal's margin is

$$0.30 - 0.1940274787 = 0.1059725213,$$

whereas the corrupted proposal exceeds the limit by

$$1.1640088764 - 0.30 = 0.8640088764.$$

3. On rejection, the adapter invokes the classical ToyHelmholtz reference solve, which has a near-machine-precision residual in the retained result. The decision is not a claim of a PhAST AT2 residual, formal safety, or a measured acceleration; it is a transparent toy fallback demonstration.

Exercise 2: Project a field and identify what DAgger still requires

For an interior node, the adapter first clamps a raw proposal to $[0, 1]$ and then applies irreversibility with $\max(d_{\text{proposal}}, d_{\text{previous}})$. Finally it enforces zero on the ToyHelmholtz boundary.

1. Compute the final value for each case: (a) raw 0.15, previous 0.25, interior; (b) raw 1.4, previous 0.25, interior; (c) raw 0.7, previous 0.25, boundary.
2. Name two interface checks that must happen before this projection.
3. The notebook writes one correction record. List the missing stages needed before it could be called an end-to-end offline DAgger cycle.

Hint 2

Apply the operations in order. The boundary rule is last, so it overrides an otherwise admissible projected value.

Worked solution 2

1. (a) Clamp leaves 0.15, irreversibility gives $\max(0.15, 0.25) = 0.25$, and the node is interior, so the final value is 0.25. (b) Clamp maps 1.4 to 1, then $\max(1, 0.25) = 1$, so the final value is 1. (c) The interior-style projection would be 0.7, but the boundary rule is applied last, so the final value is 0.
2. The adapter checks that all feature values are finite, that feature order is exactly $[x/L, y/H, \text{load factor}]$, that the mesh signature matches $[25, 13]$, and that the feature tensor shape is compatible. A feature-order mismatch or a NaN is rejected rather than silently reshaped.
3. One record is only a replay-style teaching artefact. A real offline DAgger cycle would collect states induced by the current model over a defined rollout, obtain reference labels for those states, aggregate the new labelled cases with the training data, retrain, and reassess a frozen held-out evaluation set. It should also report the reference-label cost. This lesson uses inference under **torch.no_grad()** and a ToyHelmholtz residual, so it is neither gradient-based training through PhAST nor an executed PhAST DAgger cycle.

7.9 DAgger-style data aggregation

Distribution shift can appear when a learned proposal visits states absent from its original training records. Dataset Aggregation (often called DAgger) is one way to describe an iterative collection loop:

1. train a policy or proposal model on an initial reference dataset;
2. let the current model visit stated inputs or intermediate states;
3. query a trusted reference procedure for targets at those visited states;
4. add the labelled records to the aggregate dataset; and
5. retrain while retaining an evaluation set that was not relabelled for selection.

This idea is useful when the model's own trajectory changes the data it sees. It does not remove the need for a trusted labelling procedure, provenance, or a distinction between training data and final evaluation data. In mechanics, the reference procedure may itself fail or be inappropriate outside its assumptions; record that outcome rather than silently dropping difficult cases.

7.10 Evaluation: fields, quantities, and failure modes

Field error, reaction error, and residual are related but not interchangeable. For a candidate $\widehat{d}$ and reference d, possible diagnostics include

$$e_d = \frac{\|\widehat{d} - d\|_2}{\max(\|d\|_2, \epsilon)}$$

alongside a damage-bound check and a mechanics residual after the candidate is used in the stated equation. A good evaluation card reports:

- the split and cases evaluated;
- the field metric and its weighting;
- at least one mechanics-relevant quantity;
- the frequency and criterion of correction/fallback, if applicable; and
- failures, out-of-distribution cases, or unresolved cases.

A low average field error may coexist with a wrong local crack path. A good reaction curve may coexist with a poor spatial field. Inspect both.

7.11 Exercise: identify the missing provenance

A colleague shares a file called “best_model” and says it predicts damage on a new mesh. List four items you need before interpreting the output as a mechanics field.

Solution

Any four of the following are essential: the architecture and weights; input feature definitions and ordering; normalisation state; source and target mesh representation; coordinate frame and units; load/time step represented; the training/validation/test split; a reload comparison; and the physical checks used on the proposed field. Without these, even a numerically well-formed tensor has unclear mechanical meaning. The two computational lessons provide a concrete version of this answer: the first records the model contract and reload check; the second refuses an incompatible feature order and performs a stated residual/fallback check.

7.12 Sources for this chapter

The notebook practices draw on standard reproducible machine-learning ideas: separate evaluation data, explicit preprocessing, and persistent model state. For a practical notebook pattern with open exercises and solutions, see the Inria scikit-learn MOOC. For the tensor/autodiff ecosystem used in the companion activities, see the PyTorch documentation.

CHAPTER

EIGHT

PRACTICE, SOLUTIONS, GLOSSARY, REFERENCES, AND CONTRIBUTING

8.1 A six-part learn-by-doing sequence

The book can be read in order or used as a companion to six teaching blocks. Each block should leave a visible artefact: a written prediction, a derived equation, a labelled field, a result card, or a checked derivative. Where a computational lesson appears in the table of contents beneath its chapter, read the prompt and retained output before downloading code to execute.

1. **Represent a crack.** Separate sharp cracks, cohesive interfaces, phase fields, XFEM enrichment, and nonlinear solvers.
2. **Read the energy.** Identify G_c, ℓ, $w(d)$, $g(d)$, the tension–compression split, and the damage convention.
3. **Follow an update.** Sketch a staggered mechanics/damage loop and locate the irreversibility rule.
4. **Trace data.** Start with geometry and boundary conditions; end with fields and observables; write tensor shapes at the interface.
5. **Check a derivative.** Define a scalar loss, select a direction, and compare automatic differentiation with a finite difference.
6. **Audit a learned proposal.** Save state, reload it, define the adapter, and separate a proposal from a corrected mechanics result.

The computational exercises are intentionally small. Their code, retained outputs, hints, and worked solutions belong to the same reading route as the theory. Use the timing record in the lesson rather than transferring a runtime result to a different device or software version.

8.2 Quickstart checklist

Before an activity:

- read the problem definition and prediction prompt;
- record the phase-field convention, mesh/array dimensions, and parameters;
- identify the quantity that will be checked; and
- read the corresponding integrated computational lesson; download its notebook only if you intend to execute cells in a suitable environment.

During an activity:

- change one named input at a time;
- retain the original result card when making a comparison;
- inspect a field, an observable, and a numerical check; and
- stop and diagnose if bounds, residuals, or tensor shapes are unexpected.

After an activity:

- state what was observed;
- state which assumption or numerical choice could change that observation;
- link the output to its lesson and case definition; and

- write one limitation rather than promoting a small illustration to a general material claim.

8.3 Worked consolidation exercise

Consider a two-dimensional notched specimen represented by displacement u and damage d. The calculation uses a length scale ℓ, a residual stiffness η_{res}, and a displacement-controlled load increment.

1. Give the three terms in the fracture density that regularise a sharp crack.
2. Write the qualitative order of a staggered update.
3. Name two visual outputs and one scalar output you would retain.
4. A learned model proposes d. Name two checks required before using it as a starting point for a mechanics update.
5. State one conclusion that the exercise could support and one conclusion it could not support.

Solution

1. The fracture part uses a local crack-density term $w(d)/\ell$, a gradient term $\ell|\nabla d|^2$, and the fracture energy scale G_c/c_0.
2. Start from accepted prior damage; solve mechanics with damage fixed; update the stated tensile driving quantity; solve damage subject to irreversibility; then check and accept or repeat.
3. Retain the geometry/mesh with loading direction, a damage field with a colour scale, and a displacement/deformed-field view. A suitable scalar is reaction force, an energy contribution, or a stated residual.
4. Check the damage bounds/irreversibility and a stated mechanics residual or correction step. The precise checks depend on the proposal's intended use.
5. The result can support a statement about this defined discretisation and load path. It cannot alone establish a calibrated material law, a mesh-independent crack path, or a general acceleration claim.

8.4 Glossary

Adapter
: A declared map that changes a data representation, for example node order, units, or tensor layout. It does not automatically preserve physical meaning.

Automatic differentiation (AD)
: A method for evaluating derivatives of a program by applying the chain rule to its recorded operations. Its result concerns the executed computation.

Cohesive-zone model (CZM)
: A fracture/interface formulation based on a traction–separation law.

Crack-density function $w(d)$
: The local part of a phase-field fracture regularisation. AT1 and AT2 use different choices.

Damage field d
: A scalar field used here with $d = 0$ intact and $d = 1$ fully broken.

DAgger-style aggregation
: Iterative collection of reference labels at states visited by the current proposal/policy, followed by retraining on the aggregate data.

Degradation function $g(d)$
: A function that reduces selected elastic energy as damage grows.

Finite difference
: A derivative approximation from nearby function evaluations; useful as an independent local check.

Irreversibility

The condition that brittle damage does not decrease over the stated loading history.

Matrix-free action
An implementation of $v \mapsto Kv$ or a Jacobian action without explicitly storing every global matrix entry.

Phase-field fracture
A regularised fracture formulation in which a diffuse scalar field approximates a sharp crack surface.

Quadrature
Numerical integration on an element using selected points and weights.

Quasi-Newton method
A nonlinear optimisation/solution strategy that updates an approximate Jacobian or inverse Jacobian. It is not a fracture model.

Residual
The imbalance in a stated discrete equation. A residual norm is meaningful only with its definition, scaling, tolerance, and field context.

XFEM
Extended finite elements: an enriched finite-element approximation for discontinuities or near-tip features.

8.5 Contributing a new course activity

An activity is easier for others to reuse when it has a narrow learning objective and an inspectable result. Include:

1. a one-sentence question and the prerequisite chapter;
2. a complete case definition: geometry, parameter choices, boundary conditions, and initial state;
3. a short prediction prompt before code is executed;
4. labelled figures or data products with original provenance;
5. a numerical or physical check appropriate to the task;
6. a solution or interpretation note; and
7. a limitation that prevents an over-broad claim.

Place the explanation, code, retained output, learner exercise, hint, and worked solution in that order. A standalone code download is useful for execution, but it should not become the only place where a reader can see the case definition or interpret the result.

Do not reuse a paper figure, dataset panel, or code fragment merely because it is publicly reachable. Check its licence and attribution requirements. The schematic figures in this book were authored for the course; cited papers and documentation are linked for reading rather than reproduced.

8.6 Reference trail

8.6.1 Further interactive study

Dive into Deep Learning combines exposition, mathematics, computational examples and exercises in one reading sequence. For a physics-oriented progression, Advanced Deep Learning for Physics connects numerical simulation, sensitivity analysis and learning through longer assignments. These resources informed the teaching format; the exercises and worked answers in this book were written specifically for the PhAST course.

8.6.2 Scientific references

The following sources support the formulation boundaries and teaching conventions used here.

- A. A. Griffith, "The phenomena of rupture and flow in solids," *Philosophical Transactions of the Royal Society A* 221 (1921), DOI: 10.1098/rsta.1921.0006.
- G. A. Francfort and J.-J. Marigo, "Revisiting brittle fracture as an energy minimization problem," *Journal of the Mechanics and Physics of Solids* 46 (1998), DOI: 10.1016/S0022-5096(98)00034-9.
- B. Bourdin, G. A. Francfort, and J.-J. Marigo, "Numerical experiments in revisited brittle fracture," *Journal of the Mechanics and Physics of Solids* 48 (2000), DOI: 10.1016/S0022-5096(99)00028-9.
- C. Miehe, F. Welschinger, and M. Hofacker, "Thermodynamically consistent phase-field models of fracture: Variational principles and multi-field FE implementations," *International Journal for Numerical Methods in Engineering* 83 (2010), DOI: 10.1002/nme.2861.
- K. Pham, H. Amor, J.-J. Marigo, and C. Maurini, "Gradient damage models and their use to approximate brittle fracture," *International Journal of Damage Mechanics* 20 (2011), DOI: 10.1177/1056789510386852.
- N. Moes, J. Dolbow, and T. Belytschko, "A finite element method for crack growth without remeshing," *International Journal for Numerical Methods in Engineering* 46 (1999), DOI: 10.1002/(SICI)1097-0207(19990910)46:1%3C131::AID-NME726%3E3.0.CO;2-J.
- X.-P. Xu and A. Needleman, "Numerical simulations of fast crack growth in brittle solids," *Journal of the Mechanics and Physics of Solids* 42 (1994), DOI: 10.1016/0022-5096(94)90003-5.
- P. K. Kristensen, C. F. Niordson, and E. Martínez-Pañeda, "An assessment of phase field fracture: crack initiation and growth," *Philosophical Transactions of the Royal Society A* 379 (2021), DOI: 10.1098/rsta.2021.0021.
- PyTorch contributors, Autograd documentation.
- JAX contributors, Automatic differentiation guide.
- JAX-FEM contributors, Quickstart and examples.

8.7 Final checklist

Before sharing an educational fracture result, make the reader able to answer:

- What is the crack representation?
- Which energy, convention, and constraint are used?
- Which mesh, boundary conditions, and fields are solved?
- Which quantity was observed and which check was applied?
- What is the scope of the conclusion?

If these answers are visible, the result is useful even when it is small.